

A Hybrid Algorithm for the Regular Syndrome Decoding Problem

Tianrui Wang¹, Anyu Wang^{2,7,8,9(✉)}, Kang Yang^{3(✉)}, Hanlin Liu⁴, Yu Yu^{5(✉)},
Jun Zhang^{6(✉)}, and Xiaoyun Wang^{2,7,8,9,10}

¹ Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing, China, wangtr22@mails.tsinghua.edu.cn

² Institute for Advanced Study, Tsinghua University, Beijing, China, {anyuwang,xiaoyunwang}@tsinghua.edu.cn

³ State Key Laboratory of Cryptology, Beijing, China, yangk@sklc.org

⁴ Northwestern University, Evanston, IL, USA, hanlin.liu@northwestern.edu

⁵ Shanghai Jiao Tong University, Shanghai, China, yyuu@sjtu.edu.cn

⁶ School of Mathematical Sciences, Capital Normal University, Beijing, China, junz@cnu.edu.cn

⁷ State Key Laboratory of Cryptography and Digital Economy Security, Tsinghua University, Beijing, China

⁸ School of Cryptologic Science and Engineering, Shandong University, Jinan, China

⁹ Zhongguancun Laboratory, Beijing, China

¹⁰ Shandong Institute of Blockchain, Shandong, China

Abstract. Regular Syndrome Decoding (RSD) is a variant of the traditional Syndrome Decoding (SD) problem, where the error vector is divided into consecutive, equal-length blocks, each containing exactly one nonzero element. Recently, RSD has gained significant attention due to its extensive applications in cryptographic constructions, including MPC, ZK protocols, and more. The computational complexity of RSD has primarily been analyzed using two methods: Information Set Decoding (ISD) approach and algebraic approach.

In this paper, we introduce a new hybrid algorithm for solving the RSD problem. This algorithm can be viewed as replacing the meet-in-the-middle enumeration in ISD with a process that solves quadratic equations. Our new algorithm demonstrates superior performance across a wide range of concrete parameters compared to previous methods, including both ISD and algebraic approaches, for parameter sets over both large fields ($q = 2^{128}$) and binary fields ($q = 2$). For parameter sets used in prior works, our algorithm reduces the concrete security of RSD by up to 20 bits compared to the state-of-the-art algorithms. We also provide an asymptotic analysis, identifying a broader parameter region where RSD is solvable in polynomial time compared to ISD and algebraic methods over binary fields. Additionally, we apply our algorithm to evaluate the security of the ZK protocol Wolverine (*IEEE S&P 2021*) and the OT protocol Ferret (*ACM CCS 2020*). Our results reduce the security level of Wolverine, which targets a 128-bit security level, to about 111 bits, and also marginally lowers the security of Ferret below the targeted 128-bit level for the first time.

1 Introduction

The Syndrome Decoding (SD) problem is a well-known fundamental problem in cryptography, which asks to recover a small-weight error vector $\mathbf{e}$ satisfying the linear equation $\mathbf{H} \cdot \mathbf{e} = \mathbf{s}$, for a public parity-check matrix $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$ and a public syndrome vector $\mathbf{s} \in \mathbb{F}_q^{n-k}$. This problem is known to be NP-hard [8,41] and is commonly used in the construction of cryptographic schemes.

To achieve more efficient constructions, a variant of the SD problem, known as the Regular Syndrome Decoding (RSD) problem, has garnered increasing attention recently. In the RSD problem, the error vector $\mathbf{e}$ is partitioned into consecutive blocks of equal length, with each block containing exactly one nonzero element. The use of RSD in cryptography can be traced back to the work of Augot et al. [5] in the context of the FSB hash function. More recent advancements in RSD-based constructions are primarily along the design of secure Multi-Party Computation (MPC) [27,35], Oblivious Transfer (OT) and Vector Oblivious Linear Evaluation (VOLE) in the Pseudorandom Correlation Generator (PCG) framework [13,15,44,16,42,21,1,14,10,36,9], Zero-Knowledge (ZK) proofs [42,6], signature schemes [19,22,18], etc. In some of these protocols, the RSD problem is presented in its primal form and referred to as the Learning Parity with Noise (LPN) problem with a regular noise distribution.

Due to the extensive applications of RSD, it is of substantial importance to explore the concrete hardness of the RSD problem with respect to specific parameters. Since an RSD instance can be viewed as an SD (or LPN) instance with additional constraints on the error vector, earlier studies such as [27,13] often employ algorithms for the SD and LPN problems, including Pooled Gauss [25], Information Set Decoding (ISD) [34], and Statistical Decoding [2], to estimate the concrete hardness of RSD instances. Liu et al. [31] conduct a thorough analysis of the costs associated with applying these algorithms to RSD, and suggest that advanced ISD variants, such as Stern-Dumer [39,24], MMT [32], and BJMM [7], have the lowest cost for the parameter sets used in the previous constructions [13].

The concept of ISD can be traced back to Prange’s algorithm [34], which primarily involves the repeated application of random permutations on the coordinates of $\mathbf{H}$ and $\mathbf{e}$ to obtain an equivalent SD instance $\mathbf{H}'\mathbf{e}' = \mathbf{s}$, which ensures that the first $n - k$ columns of $\mathbf{H}'$ are linearly independent (i.e., the last k coordinates form an information set), and the last k coordinates of $\mathbf{e}'$ are error-free. Then one can obtain $\mathbf{e}'$ and consequently $\mathbf{e}$ by directly solving a linear equation. Lee and Brickell [30] suggest that this method can be generalized by allowing a small number of p errors to occur in the last k coordinates of $\mathbf{e}'$, and performing an enumeration on these p errors to recover the whole $\mathbf{e}$. Advanced ISD variants, such as Stern-Dumer, MMT, and BJMM, further expedite the enumeration of the p errors using the meet-in-the-middle approach, achieving exponential speedup compared to Prange’s algorithm.

In recent works, the advanced ISD variants have been tailored to better exploit the regular structure of error vectors in the RSD problem. An approximate meet-in-the-middle approach, applicable within a specific range of parameters, is

Table 1. The selection of p in the BJMM ISD of [31] for binary RSD and in the original BJMM ISD [7] for SD. h represents the Hamming weight of the error vector $\mathbf{e}$. The choice of p in the original BJMM ISD is made for the same (n, k) , but with an error rate equal to the GV bound.

(n, k)	the h, p in [31] for RSD	the h, p in BJMM [7] for SD
$(2^{12}, 1589)$	$h = 98, p = 6$	$h \approx 0.151n, p = 228 (\approx 0.056n)$
$(2^{16}, 7391)$	$h = 389, p = 6$	$h \approx 0.305n, p = 2421 (\approx 0.037n)$
$(2^{20}, 32771)$	$h = 1419, p = 6$	$h \approx 0.396n, p = 14243 (\approx 0.014n)$
$(2^{22}, 64770)$	$h = 2735, p = 6$	$h \approx 0.427n, p = 31901 (\approx 0.008n)$

introduced in [19]. Subsequently, [26] demonstrates that the random permutation in ISD can be adapted to fit the regular structure of the error vector, achieving further acceleration. More recently, [29] improves Prange’s algorithm by incorporating partial Gaussian elimination, demonstrating enhanced efficiency for large field sizes.

The regular structure of the error vector in RSD also offers additional ways for solving the problem. A recent advancement is the algebraic approach proposed by Briaud and Øygarden [17], which leverages the regular structure to construct quadratic equation system and applies the XL algorithm [20] alongside Gröbner basis techniques to solve it. This algebraic method has been shown to outperform ISD algorithms for a class of low-code-rate RSD parameters.

Motivation and Observation. Advanced ISD algorithms are generally considered the most effective methods for solving the original SD problem. However, as suggested by the results of [17], these methods exhibit suboptimal performance for certain RSD parameters. Our key observation is that this inefficiency arises because the optimal selection of p in ISD algorithms is often small for RSD applications, causing these algorithms to closely resemble Prange’s algorithm.

Specifically, for large field sizes such as $q = 2^{128}$, the optimal choice of p degenerates to 0 when applying ISD algorithms to solve RSD (as demonstrated using the scripts of [31]), effectively reducing the process to Prange’s algorithm. This issue also arises in the binary case for low error rates. For example, consider the parameter set in Table 1, which is analyzed in [31] for applying BJMM ISD to RSD. Here, the optimal choice of p is 6 for n ranging from 2^{12} to 2^{22} , whereas the original BJMM ISD suggests a much larger p for the same code rate and an error rate matching the Gilbert-Varshamov (GV) bound [7]. This phenomenon also occurs for slightly larger error rates [26]. A detailed analysis is provided in Supplementary Material A.

Given that meet-in-the-middle techniques provide the only exponential acceleration for advanced ISD variants compared to Prange’s algorithm, we argue that such a small p for RSD parameters fails to fully exploit ISD’s potential, leaving substantial room for optimization.

Table 2. The time complexity of the algebraic method [17], the ISD algorithms [31,26,29], and our hybrid algorithm for the computation of RSD over **large fields** with $q = 2^{128}$. The parameter sets are sourced from [31,26,17,29], where h represents the Hamming weight of the error vector $\mathbf{e}$. All complexities count the number of field operations and are expressed logarithmically with base 2, and the number highlighted in bold is the best complexity among these algorithms. The abbreviation “AGB” refers to the algebraic method in [17], “RP” refers to the Reduce and Prange method in [29].

RSD(n, k, h)	AGB in [17]	ISD in [31]	Regular-ISD in [26]	RP in [29]	Ours
$(2^{22}, 64770, 4788)$	110	152	173	151	89.27
$(2^{20}, 32771, 2467)$	131	155	173	148	114.27
$(2^{18}, 15336, 1312)$	133	153	160	147	119.03
$(2^{16}, 7391, 667)$	150	151	160	142	131.94
$(2^{14}, 3482, 338)$	150	150	153	139	133.15
$(2^{12}, 1589, 172)$	150	155	148	140	132.60
$(2^{10}, 652, 106)$	179	194	173	159	158.21
$(2^{22}, 64770, 2735)$	104	108	126	105	86.51
$(2^{20}, 32771, 1419)$	102	107	122	103	83.91
$(2^{18}, 15336, 760)$	107	105	117	100	90.95
$(2^{16}, 7391, 389)$	111	103	113	97	96.39
$(2^{14}, 3482, 198)$	110	101	107	94	95.62
$(2^{12}, 1589, 98)$	107	100	101	89	93.08
$(2^{10}, 652, 57)$	111	111	106	98	97.61

1.1 Our Technique and Result

To address the degeneration phenomenon observed when applying advanced ISD variants to RSD, we propose substituting the meet-in-the-middle enumeration in ISD with a more efficient approach to partially recover the error vector (i.e., the last k coordinates of $\mathbf{e}'$). Broadly speaking, we construct quadratic equations for the variables corresponding to the last k coordinates of $\mathbf{e}'$ by leveraging the structure of the error vector in RSD, and then solve these quadratic equations using a modified hybrid method.

The hybrid method, which includes algorithms such as the Crossbred algorithm [28] and the BDT algorithm [12], combines techniques such as partial guessing of variables, exhaustive enumeration, and local state refreshing. We find that the structure of the error vector in RSD allows us to guess the positions of nonzero elements rather than their exact values, significantly reducing the computational complexity of solving quadratic equations, especially for large field sizes. This advantage provides a better parameter trade-off compared to the degeneration trend observed in advanced ISD variants, resulting in a more effective approach for RSD.

Table 3. The time complexity of the algebraic method [17], the ISD algorithms [31,26,29], and our proposed hybrid algorithm for the computation of **binary RSD**.

RSD(n, k, h)	AGB in [17]	ISD in [31]	Regular-ISD in [26]	RP in [29]	Ours
$(2^{22}, 64770, 4788)$	103	135	140	143	87.04
$(2^{20}, 32771, 2467)$	125	135	138	139	105.44
$(2^{18}, 15336, 1312)$	122	129	126	137	108.27
$(2^{16}, 7391, 667)$	139	125	126	130	119.67
$(2^{14}, 3482, 338)$	138	122	118	125	118.53
$(2^{12}, 1589, 172)$	135	121	109	120	113.69
$(2^{10}, 652, 106)$	145	159	113	117	115.58
$(2^{22}, 64770, 2735)$	103	99	103	102	86.92
$(2^{20}, 32771, 1419)$	98	97	98	100	81.09
$(2^{18}, 15336, 760)$	104	92	93	97	86.74
$(2^{16}, 7391, 389)$	108	87	90	92	90.90
$(2^{14}, 3482, 198)$	106	83	84	88	88.97
$(2^{12}, 1589, 98)$	103	80	78	84	82.16
$(2^{10}, 652, 57)$	101	90	76	84	78.43

Specifically, our algorithm consists of two main parts.

(1) We randomly select error-free positions of a regular error vector $\mathbf{e}$ and remove the corresponding columns and positions from $\mathbf{H}$ and $\mathbf{e}$, yielding an equivalent RSD instance $\mathbf{H}'\mathbf{e}' = \mathbf{s}$.¹ Treating the components of $\mathbf{e}'$ as variables, we can use the linear relationship $\mathbf{H}'\mathbf{e}' = \mathbf{s}$ to express some variables in $\mathbf{e}'$ as linear combinations of the others. Then, leveraging the fact that the product of any two distinct variables within the same block is zero, which is a natural algebraic description for the regular property of the error vector, we construct a quadratic equation system. This part is analogous to applying a random permutation and performing Gaussian elimination in ISD, with the key difference that while ISD transforms the decoding problem into a column-matching problem solved via meet-in-the-middle techniques, our method reformulates it as an algebraic equation-solving problem.

(2) We solve the quadratic equations using the hybrid method. Specifically, we partition the variables in the quadratic system into *unknown* variables and *coefficient* variables, and then guess a subset of the coefficient variables, resulting in a well-constrained polynomial system primarily involving the unknown variables, which can be solved directly. The regular structure of the error vector significantly accelerates the process of guessing the coefficient variables. Instead of guessing the exact values of each variable, we only guess the positions of the

¹ The error vector $\mathbf{e}'$ is not strictly ‘regular’, but it can still be divided into unequal blocks, each containing at most one nonzero element.

Table 4. The time complexity of applying different methods to Wolverine [42] and Ferret [44]. The parameter sets are suggested in Wolverine and Ferret for 128-bit security.

Protocol	$\text{RSD}(n, k, h)$	AGB in [17]	ISD in [31]	Regular-ISD in [26]	RP in [29]	Ours
Wolverine	(642048, 19870, 2508)	126	154	172	147	110.78
	(10805248, 589760, 1319)	165	160	177	155	149.48
Ferret	(609728, 36288, 1269)	143	138	140	144	126.59
	(10805248, 589760, 1319)	164	152	157	155	149.90

nonzero variables within each block, which provides a substantial speedup for large q . Additionally, we incorporate the preprocessing technique proposed in [12], achieving a polynomial speedup in n . Given that n is typically large in RSD parameters (e.g., $n = 2^{20}$), this speedup is highly impactful in practice.

We demonstrate that our new algorithm facilitates advancements in the complexity of solving the RSD problem in certain cases. Specifically, we conduct comparisons with the algebraic approach from [17] and the ISD approaches from [31, 26, 29] using the previously adopted parameter sets. These comparisons cover both the large field case ($q = 2^{128}$) and the binary case ($q = 2$), as shown in Table 2 and Table 3, respectively. The results indicate that our hybrid algorithm reduces the concrete hardness of RSD over both large and binary fields, with improvements of up to 20 bits compared to the state-of-the-art algorithms. Furthermore, our algorithm outperforms the algebraic approach in [17] for all listed parameters. It also surpasses the ISD approaches [31, 26, 29] for relatively small code rates k/n (corresponding to larger values of n in Table 2 and Table 3).

We further demonstrate the effectiveness of our algorithm by applying it to parameter sets used in practical constructions, such as Wolverine [42] and Ferret [44]. Wolverine is a protocol for constant-round interactive ZK proofs, while Ferret is a practical efficient oblivious transfer extension protocol that has been included in multiple open-sourced libraries like EMP-toolkit [40], PCF 2.0 [33] and SecretFlow [4]. As depicted in Table 4, our algorithm surpasses both the algebraic method [17] and the ISD algorithms [31, 26, 29] for parameter sets targeting 128-bit security in both Wolverine and Ferret. Moreover, for the parameter set with $n = 642048$ in Wolverine, our algorithm reduces the current optimal complexity of the analysis algorithm for this parameter from 126 bits to 111 bits. For the parameter set with $n = 609728$ in Ferret, our algorithm indicates that the concrete security of binary RSD is marginally below the targeted 128-bit security level for the first time.

Additionally, we present an analysis of the asymptotic complexity of our hybrid algorithm. For binary RSD, our analysis shows that the total time complexity of the algorithm is at most polynomial in n if the parameters satisfy either $R \leq \rho$, or $R > \rho$ and $(R - \rho)^2 \rho \leq n^{-1}$, where $R = k/n$ is the code rate and $\rho = h/n$ is the error rate. While regions leading to polynomial time

complexity have also been established for the algebraic approach in [17] and ISD algorithms in [26], our result suggests broader parameter regions than those previously established. Moreover, we compare the asymptotic complexity of the algebraic approach with our method across different parameter categories. The results demonstrate that our algorithm tends to outperform the algebraic approach for constant code rate and $\rho = o(1)$, while the algebraic approach is more efficient when $R = o(1)$ and $\rho = \lambda R$ with a constant $\lambda < 1$ or R, ρ are constants.

1.2 Related Work

As mentioned earlier, there are two types of highly effective algorithms for solving RSD. One is the ISD algorithms [31, 26, 29], and the other is the algebraic method introduced in [17].

ISD Algorithms. To our knowledge, the most advanced ISD algorithms for RSD are proposed by Liu et al. [31], Esser and Santini [26], Kim and Lee [29]. In [31], the authors employ additional parity checks to reduce the problem dimension. In [26], the authors replace the random permutation with a ‘regular’ permutation in each block and enumerate p -weight vectors using the regular structure of the error vector (which they called Regular-ISD). Both of the previous works primarily target binary RSD and degenerate into a Prange-like algorithm when the field size is large. In [29], the authors enhance the performance of ISD by reducing the cost associated with the Gaussian elimination process when the field size is large. Our hybrid approach differs from the ISD approach in determining the error vector by solving quadratic equations introduced by its regular structure, rather than using the meet-in-the-middle enumeration.

Algebraic Method. The algebraic approach proposed by Briaud and Øygarden [17] introduces the technique of Gröbner bases to solve the polynomial system and is known as the first non-ISD algorithm that outperforms ISD algorithms on concrete RSD parameters especially when the code rate k/n is small. Our approach differs from the algebraic approach [17] in the handling of quadratic equations. Furthermore, concrete comparisons demonstrate that our hybrid algorithm outperforms the algebraic approach across a wide range of parameters.

1.3 Organizations

Section 2 provides the notations and preliminary concepts used throughout this paper. Section 3 presents our new hybrid approach for RSD over arbitrary fields. Section 4 analyzes the complexity of our approach and its performance on concrete parameters, along with an analysis of the asymptotic complexity. Section 5 details the optimizations for binary RSD and the corresponding complexity analysis. Section 6 examines the performance of our algorithm on two concrete protocols, Wolverine and Ferret. Section 7 concludes the paper.

2 Preliminary

2.1 Notation

- For a vector $\mathbf{y} = (y_1, \dots, y_n)^T \in \mathbb{F}_q^n$, we denote $w_H(\mathbf{y})$ as the Hamming weight of $\mathbf{y}$, which represents the number of nonzero positions in $\mathbf{y}$.
- For a matrix $\mathbf{H} \in \mathbb{F}_q^{m \times n}$, we denote $\mathbf{H}_{i,j}$ as the element in the i -th row and j -th column of $\mathbf{H}$, where $1 \leq i \leq m$ and $1 \leq j \leq n$. We denote $\mathbf{I}_m$ as the $m \times m$ identity matrix. We use $\text{Rank}(\mathbf{H})$ to denote the rank of $\mathbf{H}$ over $\mathbb{F}_q$.
- For variables $x_1, \dots, x_n$, we denote $\mathbb{F}_q[x_1, \dots, x_n]$ as their polynomial ring over $\mathbb{F}_q$.

2.2 The RSD Problem

Definition 1 (Computational RSD Problem). Suppose that n, k, h and $\beta = n/h$ are positive integers. Given a parity check matrix $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$, a syndrome vector $\mathbf{s} \in \mathbb{F}_q^{n-k}$, then $\text{RSD}(n, k, h)$ aims to find an error vector $\mathbf{e} = (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_h)$ such that $\mathbf{H}\mathbf{e} = \mathbf{s}$, $w_H(\mathbf{e}_i) = 1$ and $\mathbf{e}_i \in \mathbb{F}_q^\beta$ for $1 \leq i \leq h$.

Each entry of the error vector $\mathbf{e}$ is referred to as a *variable*, and the indices corresponding to the variables of each $\mathbf{e}_i$ are termed as a *block*.

2.3 Algebraic Methods for Solving Polynomial Equation Systems

The Hybrid Method. As discussed earlier, this paper focuses on hybrid approaches for solving polynomial equation systems. These methods employ various techniques to solve equations, achieving relatively low complexity through parameter trade-offs. Notable examples include the Crossbred algorithm [28] and the BDT algorithm [12]. While the Crossbred algorithm handles polynomial systems of general degree d , the BDT algorithm specializes in quadratic systems ($d = 2$) and can be viewed as an optimized variant of Crossbred.

In this work, we focus exclusively on the BDT algorithm because we observe that the hybrid method for $d > 2$ degenerates into a process of guessing all variables rather than solving equations for most RSD parameters. As a result, the complexity for $d > 2$ becomes impractical for concrete parameter settings.

Given m quadratic equations and n variables, the BDT algorithm partitions the variables into v *unknown* variables and u *coefficient* variables, requiring $m \geq \frac{v^2+v}{2}$. This partitioning creates an overdetermined system, enabling the elimination of quadratic terms involving the v unknown variables via Gaussian elimination. The algorithm then enumerates the u coefficient variables, locally updates the equations by substituting these variables, and solves the resulting linear system in polynomial time.

Other Approaches. Besides hybrid methods, other approaches exist for solving polynomial equation systems. The first is the Fast Exhaustive Search (FES) approach [11], an enhanced version of exhaustive search, which has a complexity of $\mathcal{O}(d \log_2(n) 2^n)$, where d is the degree and n is the number of variables. The second method, proposed by Williams [43], represents the system as a single low-degree probabilistic polynomial equation. The best algorithm in this category is given by Dinur [23], with a complexity of $\mathcal{O}(n^2 2^{0.815n})$. The third category is Gröbner basis methods, such as the XL algorithm used in [17], which represents the system using Gröbner bases and solves the corresponding Macaulay matrix.

Why Use the Hybrid Method? A common intuition is that a larger number of equations simplifies solving polynomial systems. For instance, the hardest case arises when $m \approx n$, where m is the number of equations and n the variables. In RSD problems, the number of quadratic equations satisfies $m = \mathcal{O}(\beta^2 h) = \mathcal{O}(\beta n)$, with β approximating a constant. This surplus of equations provides potential for simplifying the solving process. However, most methods are designed to minimize complexity in the worst case, i.e. $m \approx n$. For example, both FES and the polynomial approach achieve asymptotic complexity $\mathcal{O}(2^{\alpha n})$ for a constant α . These methods fail to leverage the surplus of equations, making them impractical for RSD applications where n typically ranges from 10^3 to 10^4 . Gröbner basis methods, as in [17], construct a high-degree Macaulay matrix and solve it using the Wiedemann algorithm (a sparse matrix Gaussian elimination variant). However, the surplus of equations often results in a low-degree Macaulay matrix (e.g., $d = 2$ or 3), causing the algorithm to degenerate with limited benefits from sparsity. Thus, we argue the hybrid method is better suited for RSD applications.

Moreover, our hybrid algorithm does not simply replace the BDT subroutine in the algebraic method but instead strategically leverages the system’s regular structure. To validate this, we compare three approaches under fixed parameters: (1) the algebraic method in [17], (2) a direct substitution replacing XL with BDT from [12], and (3) our hybrid algorithm. Detailed results are provided in Supplementary Material B.

2.4 Complexity of Gaussian Elimination

When performing Gaussian elimination on a matrix $\mathbf{A} \in \mathbb{F}_g^{n \times n}$, a straightforward implementation yields a time complexity of $T_{\text{Gauss}} = \mathcal{O}(n^3)$. Advanced methods can achieve better complexity. For example, Strassen’s algorithm reduces the complexity of Gaussian elimination to $\mathcal{O}(n^\omega)$, where $\omega \approx 2.8$. This approach has been utilized in [31, 28] for analyzing the complexity of RSD. For Gaussian elimination over the binary field, [3] proposes an algorithm using fast linear algebra techniques, which achieves a complexity of $T_{\text{Gauss}} = \mathcal{O}(n^3 / \log_2 n)$ or $\mathcal{O}(n^2 / \log_2 n)$ row operations for non-square matrices.

3 The Hybrid Algorithm for RSD over $\mathbb{F}_q$

The process of the algorithm is summarized in Algorithm 1 and can be divided into two stages. The first stage, consisting of steps 1-3, aims to construct a quadratic equation system with reduced number of variables, which is specified in Section 3.1. The second stage, outlined in step 4, aims to solve the quadratic equation system via a hybrid approach, which is specified in Section 3.2.

Algorithm 1: Hybrid algebraic algorithm for RSD(n, k, h)

Input: $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$ and $\mathbf{s} \in \mathbb{F}_q^{n-k}$ for an RSD(n, k, h) problem
Input: Parameters $f, g, u \in \mathbb{Z}$ such that $f, g, u \geq 0$, $f + g \leq h$ and $u \leq \beta - 1$
Output: A solution $\mathbf{e} \in \mathbb{F}_q^n$ for the RSD(n, k, h) problem

- 1: Randomly select $f \cdot u$ positions and remove the corresponding columns and variables from $\mathbf{H}$ and $\mathbf{e}$ to derive a new equation $\mathbf{H}'\mathbf{e}' = \mathbf{s}$ of reduced dimension.
 $\triangleright$ Select u positions from each of the first f blocks.
- 2: Apply Gaussian elimination to transform $\mathbf{H}'\mathbf{e}' = \mathbf{s}$ into $[\mathbf{I}_{n-k} || \mathbf{Q}] \cdot \mathbf{e}' = \mathbf{T}\mathbf{s}$.
 $\triangleright$ $\mathbf{T}$ is an invertible matrix s.t. $\mathbf{T}\mathbf{H}' = [\mathbf{I}_{n-k} || \mathbf{Q}]$.
- 3: Substitute $[\mathbf{I}_{n-k} || \mathbf{Q}] \cdot \mathbf{e}' = \mathbf{T}\mathbf{s}$ into the quadratic equations $e_{i,j_1}e_{i,j_2} = 0$ to eliminate the first $(n-k)$ variables of $\mathbf{e}'$, and obtain a quadratic equation system with reduced number of variables.
 $\triangleright$ $e_{i,j_1}e_{i,j_2} = 0$ for distinct e_{i,j_1}, e_{i,j_2} in the same block.
- 4: Apply hybrid method to enumerate the nonzero positions of $g\beta$ variables and solve the quadratic equation system to obtain the solutions for the last $k - fu$ variables of $\mathbf{e}'$.
 $\triangleright$ An additional condition for f, g, u is required, as specified in (9).
- 5: Compute $\mathbf{e}'$ using $[\mathbf{I}_{n-k} || \mathbf{Q}] \cdot \mathbf{e}' = \mathbf{T}\mathbf{s}$, and recover $\mathbf{e}$ by filling the fu positions with 0's.
 $\triangleright$ A correct $\mathbf{e}$ can be recovered if the removed fu positions are all 0's.
- 6: **If** $\mathbf{e}$ is a solution for the RSD(n, k, h) problem **then**
- 7: **return** $\mathbf{e}$;
- 8: **Else**
- 9: **goto** line 1.
- 10: **Endif**

3.1 Construct the Quadratic Equations via Gaussian Elimination

This stage aims to transform the RSD(n, k, h) into a system of quadratic equations. It is important to note that the techniques used to guess error-free positions and transform the regular structure into quadratic equations are analogous to those in [17]. However, our goal at this stage is to derive an equation system with fewer variables that can be solved efficiently using the hybrid algebraic method. This differs from the approach in [17], where the Gröbner basis is used directly to solve the quadratic equation system formed from the regular structure.

Guess Error-Free Positions. In this step, we randomly select u columns from each of the first f blocks of $\mathbf{H}$ to form a new matrix $\mathbf{H}' \in \mathbb{F}_q^{(n-k) \times (n-fu)}$ by removing these fu columns. We define $\mathbf{e}' = (\mathbf{e}'_1, \dots, \mathbf{e}'_h)$, where $\mathbf{e}'_i$ is obtained by puncturing the corresponding u variables of $\mathbf{e}_i$ for $1 \leq i \leq f$, and $\mathbf{e}'_i = \mathbf{e}_i$ for $f < i \leq h$. Next, we focus on the equation

$$\mathbf{H}'\mathbf{e}' = \mathbf{s}. \quad (1)$$

It is evident that if the fu variables punctured from $\mathbf{e}_i$ are all 0's, then a solution to (1) can be extended to recover $\mathbf{e}$ by filling in the fu positions with 0's.

To deduce the probability that the fu variables in $\mathbf{e}$ are all 0, we assume a priori that $\Pr(e_{i,j} = 0) = 1 - \frac{1}{\beta}$ and $\Pr(e_{i,j} \neq 0) = \frac{1}{\beta}$ for $1 \leq i \leq h$ and $1 \leq j \leq \beta$ in the $\text{RSD}(n, k, h)$. Thus, the probability that these fu positions are error-free is given by

$$P_{f,u} := \left(1 - \frac{u}{\beta}\right)^f. \quad (2)$$

Gaussian Elimination. This step transforms (1) into the systematic form. Specifically, let $\mathbf{T} \in \mathbb{F}_q^{(n-k) \times (n-k)}$ be an invertible matrix such that $\mathbf{T} \cdot \mathbf{H}' = [\mathbf{I}_{n-k} \parallel \mathbf{Q}]$. Then, (1) can be transformed into

$$[\mathbf{I}_{n-k} \parallel \mathbf{Q}] \cdot \mathbf{e}' = \mathbf{T}\mathbf{s}. \quad (3)$$

Note that it is possible that the first $(n-k)$ columns of $\mathbf{H}'$ are linearly dependent, which would render the elimination infeasible. A similar issue can arise during the partial elimination step in ISD-type algorithms, such as those in [31, 26, 29]. This issue is generally not handled explicitly, as the probability of linear dependence is at most a constant for a random matrix. In our algorithm, we aim to address RSD instances involving random codes as well as other types of codes, such as the local linear codes used in [44]. To this end, we conduct experiments to assess the Gaussian elimination process for RSD dimension n ranging from 500 to 7000, and demonstrate that the probability of linear dependence tends to a constant for both random codes and local linear codes as the matrix size increases. The details are provided in Supplementary Material C.1.

Construct Quadratic Equations. Note that the vector $\mathbf{e}' = (\mathbf{e}'_1, \dots, \mathbf{e}'_h)$ satisfies the condition $w_H(\mathbf{e}'_i) = 1$ for $1 \leq i \leq h$. Similar to that in [17] we can construct

$$m_0 := f \cdot \binom{\beta - u}{2} + (h - f) \cdot \binom{\beta}{2}$$

quadratic equations of the form $e'_{i,j_1} \cdot e'_{i,j_2} = 0$ for all variables e'_{i,j_1}, e'_{i,j_2} that are in the same block of $\mathbf{e}'$.

On the other hand, write $\mathbf{e}' = (\mathbf{e}^{(1)} \parallel \mathbf{e}^{(2)})$ such that $\mathbf{e}^{(1)} \in \mathbb{F}_q^{n-k}$ and $\mathbf{e}^{(2)} \in \mathbb{F}_q^{n_0}$, where $n_0 := k - fu > 0$. Then by (3) it has the following linear equations

$$\mathbf{e}^{(1)} = -\mathbf{Q}\mathbf{e}^{(2)} + \mathbf{T}\mathbf{s}. \quad (4)$$

Substituting these linear equations into the m_0 quadratic equations $e'_{i,j_1} \cdot e'_{i,j_2} = 0$ to eliminate the variables of $\mathbf{e}^{(1)}$, we can obtain a new quadratic equation system with respect to the n_0 variables of $\mathbf{e}^{(2)}$.

By regarding all the quadratic terms and linear terms of these n_0 variables as new variables, we can write the quadratic equation system in the form

$$\mathbf{W}\mathbf{x} = \mathbf{d}, \quad (5)$$

where $\mathbf{W} \in \mathbb{F}_q^{m_0 \times (n_0^2 + 3n_0)/2}$ and $\mathbf{d} \in \mathbb{F}_q^{m_0}$. Here the $(n_0^2 + 3n_0)/2$ variables of $\mathbf{x}$ correspond to the monomials of the variables of $\mathbf{e}^{(2)}$ with degrees 1 and 2, i.e., n_0 variables of degree 1 and $(n_0^2 + n_0)/2$ variables of degree 2. Note that the matrix $[\mathbf{W} \parallel -\mathbf{d}]$ is actually the degree 2 Macaulay matrix with respect to the variables of $\mathbf{e}^{(2)}$.

3.2 Solve the Quadratic Equations by the Hybrid Method

In this stage, our goal is to solve the equation system of (5) by the hybrid algebraic method as mentioned in Section 2.3. To optimize this process, we adapt the enumeration strategy to more effectively accommodate the regular structure. Furthermore, we adopt the preprocessing technique proposed in [12] for quadratic equation systems to facilitate a polynomial-level speedup.

Preprocess the Equation System. Denote $w := g\beta$ and $v := n_0 - w$, and write the n_0 variables in $\mathbf{e}^{(2)}$ as $(\tilde{e}_1, \dots, \tilde{e}_v, \tilde{e}_{v+1}, \dots, \tilde{e}_{v+w})$ such that $(\tilde{e}_{v+1}, \dots, \tilde{e}_{v+w})$ contains exactly g blocks. We call the v variables as *unknown* variables and the w variables as *coefficient* variables. Then we will enumerate parts of the coefficient variables and solve the quadratic equations of unknown variables and the remaining coefficient variables. By interpreting (5) as equations over the polynomial ring $\mathbb{F}_q[\tilde{e}_{v+1}, \dots, \tilde{e}_{v+w}]$, we can rewrite (5) as

$$\mathbf{W}'\mathbf{x}' = \mathbf{d}', \quad (6)$$

where $\mathbf{W}'$ is an $m_1 \times (v^2 + 3v)/2$ matrix, $\mathbf{d}'$ is a length m_1 vector over the polynomial ring $\mathbb{F}_q[\tilde{e}_{v+1}, \dots, \tilde{e}_{v+w}]$, and

$$m_1 := m_0 - g \cdot \binom{\beta}{2} = f \binom{\beta - u}{2} + (h - f - g) \binom{\beta}{2}.$$

The $(v^2 + 3v)/2$ variables of $\mathbf{x}'$ correspond to the monomials

$$\{\tilde{e}_i \tilde{e}_j\}_{1 \leq i < j \leq v} \cup \{\tilde{e}_i^2\}_{1 \leq i \leq v} \cup \{\tilde{e}_i\}_{1 \leq i \leq v}.$$

The number of rows of $\mathbf{W}'$ is reduced because the $g \cdot \binom{\beta}{2}$ equations involving only the variables $\tilde{e}_{v+1}, \dots, \tilde{e}_{v+w}$ are removed.

Since the equations in (5) are all quadratic, the $\binom{v}{2} + v$ columns of $\mathbf{W}'$ corresponding to the monomials $\{\tilde{e}_i \tilde{e}_j\}_{1 \leq i < j \leq v}$ and $\{\tilde{e}_i^2\}_{1 \leq i \leq v}$ are still in $\mathbb{F}_q$, and

each entry of the other v columns of $\mathbf{W}'$ has degree no more than 1. Assuming the order of variables is arranged appropriately, $\mathbf{W}'$ can be written in the form

$$\mathbf{W}' = [\mathbf{W}'_1 \parallel \mathbf{W}'_2],$$

where $\mathbf{W}'_1 \in \mathbb{F}_q^{m_1 \times v(v+1)/2}$ is a matrix in $\mathbb{F}_q$, and each entry of $\mathbf{W}'_2$ is an $\mathbb{F}_q$ -linear combination of $\tilde{e}_{v+1}, \dots, \tilde{e}_{v+w}$ and 1. We can then perform partial Gaussian elimination to obtain an invertible matrix $\mathbf{T}' \in \mathbb{F}_q^{m_1 \times m_1}$ such that

$$\mathbf{T}'\mathbf{W}' = \begin{bmatrix} \mathbf{I}_{v(v+1)/2} & \mathbf{W}'_2{}^{(0)} \\ \mathbf{O} & \mathbf{W}'_2{}^{(1)} \end{bmatrix} \text{ and } \mathbf{T}'\mathbf{d}' = \begin{bmatrix} \mathbf{d}'^{(0)} \\ \mathbf{d}'^{(1)} \end{bmatrix}. \quad (7)$$

Thus by (6) we have

$$\mathbf{W}'_2{}^{(1)} \cdot \begin{bmatrix} \tilde{e}_1 \\ \vdots \\ \tilde{e}_v \end{bmatrix} = \mathbf{d}'^{(1)}, \quad (8)$$

where $\mathbf{W}'_2{}^{(1)}$ is an $(m_1 - v(v+1)/2) \times v$ matrix and $\mathbf{d}'^{(1)}$ is a length $(m_1 - v(v+1)/2)$ vector over $\mathbb{F}_q[\tilde{e}_{v+1}, \dots, \tilde{e}_{v+w}]$.

Note that $\text{Rank}(\mathbf{W}'_1)$ should be $v(v+1)/2$ to ensure that the partial Gaussian elimination can proceed. To ensure that Gaussian elimination succeeds with overwhelming probability, the parameters in our algorithm are chosen such that $\mathbf{W}'_1$ has far more rows than columns. The specific condition is outlined later in (9), as it is related to solving equation (8).

Remark 1. The rows of $\mathbf{W}'_1$ are derived by expanding the pairwise products of the rows of $\mathbf{Q}\mathbf{e}^{(2)} + \mathbf{T}\mathbf{s}$ in (4) and thus are not random. However, we argue that this process does not inherently introduce linear dependence among the rows of $\mathbf{W}'_1$. In fact, since $\mathbf{Q}$ is obtained through Gaussian elimination of the matrix $\mathbf{H}'$, it can be reasonably viewed as a random matrix. In general, for a set of t linearly independent combinations of the variables in $\mathbf{e}^{(2)}$ over $\mathbb{F}_q$, their $t(t+1)/2$ pairwise products are also linearly independent over $\mathbb{F}_q$. Thus, we heuristically assume that the $\mathbf{W}'_1$ is full rank if it has far more rows than columns. In fact, performing Gaussian elimination on the Macaulay matrix is a common issue when solving polynomial equation systems via hybrid approaches [28,12], which is typically addressed by ensuring the matrix has more rows than columns. On the other hand, while Gaussian elimination is not used in the algebraic method for RSD in [17], they rely on a heuristic assumption of semi-regular systems, which also requires a form of independence among the equations for any degree lower than the degree of regularity. For the sake of conservativeness, we conduct experiments to assess the partial Gaussian elimination process of $\mathbf{W}'_1$ for concrete RSD instances. The size of $\mathbf{W}'_1$ ranges from 198×171 to 28559×28203 , and we observe no failures in this step. The details are provided in Supplementary Material C.2.

Partially Enumerate Variables and Solve Equation (8). Given that the coefficient variables $(\tilde{e}_{v+1}, \dots, \tilde{e}_{v+w})$ contain exactly g blocks, a straightforward approach to solving (8) is to first enumerate the $\beta^g(q-1)^g$ possible values for the vector $(\tilde{e}_{v+1}, \dots, \tilde{e}_{v+w})$. This transforms (8) into a system of linear equations, which can then be solved directly. However, since the field size q may be very large (e.g., $q = 2^{128}$), enumerating all possible values of the $g\beta$ variables can be computationally infeasible. To address this, we instead enumerate only the positions of the g nonzero elements within each block, leaving their values undetermined. This reduces the enumeration cost to β^g . After enumeration, we denote the remaining g coefficient variables as $\tilde{e}_{v+a_1}, \dots, \tilde{e}_{v+a_g}$, where $1 \leq a_1 < \dots < a_g \leq w$. This transforms (8) into a quadratic equation system with respect to the coefficient variables $\tilde{e}_{v+a_1}, \dots, \tilde{e}_{v+a_g}$ and unknown variables $\tilde{e}_1, \dots, \tilde{e}_v$. Furthermore, the monomials in the system can only take the forms:

$$\{\tilde{e}_i\}_{1 \leq i \leq v} \cup \{\tilde{e}_{a_i}\}_{1 \leq i \leq g} \cup \{\tilde{e}_{a_i}^2\}_{1 \leq i \leq g} \cup \{\tilde{e}_i \tilde{e}_{a_j}\}_{1 \leq i \leq v, 1 \leq j \leq g} \cup \{\tilde{e}_{a_i} \tilde{e}_{a_j}\}_{1 \leq i < j \leq g}.$$

Let $z = v + 2g + gv + \binom{g}{2}$ denote the number of such monomials. By constructing the corresponding Macaulay matrix, we can treat (8) as a linear equation system with $m_1 - v(v+1)/2$ equations and z variables. To ensure the system is well-constrained, we impose the condition

$$m_1 - \frac{v(v+1)}{2} \geq \frac{5z}{2} = \frac{5}{2} \left(v + 2g + gv + \binom{g}{2} \right), \quad (9)$$

which guarantees that the number of equations significantly exceeds the number of variables. Consequently, the values of $\{\tilde{e}_1, \dots, \tilde{e}_v, \tilde{e}_{a_1}, \dots, \tilde{e}_{a_g}\}$ (which belong to the z monomials) can be determined via Gaussian elimination. If the fu positions selected in Section 3.1 are error-free, the correct vector $\mathbf{e}^{(2)}$ must satisfy (8) and will thus be enumerated by our approach.

Remark 2. One might be interested in the choice of $5z/2$. In fact, this is because we adopt a conservative approach to parameter selection, following the choice of $5/2$ in [12] to ensure that the linear equation system is well-constrained. However, other values greater than 1 could be used for a more aggressive parameter choice.

3.3 Further Optimizations

In this subsection, we introduce two optimizations that can be applied to accelerate Algorithm 1. These optimizations are relatively simple and typically yield polynomial speedups. However, they are particularly effective in reducing the complexity of solving RSD, as concrete RSD instances often involve very large dimensions n .

Evenly Select the fu Positions from Each Block. Observe that for a fixed product $S := fu$, a larger f and smaller u correspond to a higher probability $P_{f,u} = (1 - u/\beta)^f$. Thus, for a fixed number of positions to be punctured in

Section 3.1, a natural optimization strategy to increase $P_{f,u}$ is to set $f' = h - g$ and $u' = S/f'$, that is, to select the positions to be punctured evenly across the $h - g$ blocks. However, for specific RSD parameters, S/f' often does not result in an integer.

This inspires us to adjust our selection of error-free positions such that $\bar{u} + 1$ positions are chosen from each of the $\bar{f}$ blocks, and $\bar{u}$ positions are chosen from each of the $h - g - \bar{f}$ blocks. Although the number of equations m_1 in (7) can be affected by this adjustment, the increase in the probability $P_{f,u}$ can be substantial for certain parameter. For this new strategy, it can be deduced that the probability that the selected positions being error-free is

$$P'_{\bar{f},\bar{u}} = \left(1 - \frac{\bar{u} + 1}{\beta}\right)^{\bar{f}} \left(1 - \frac{\bar{u}}{\beta}\right)^{(h-g-\bar{f})} \quad (10)$$

and the number of equations in (7) becomes

$$m_1 = \bar{f} \binom{\beta - \bar{u} - 1}{2} + (h - g - \bar{f}) \binom{\beta - \bar{u}}{2}. \quad (11)$$

Besides, the total number of columns deleted is $(h - g)\bar{u} + \bar{f}$, and thus the number of variables of $\mathbf{e}^{(2)}$ in (4) becomes

$$n_0 = k - \bar{f} - (h - g)\bar{u}.$$

Other stages follow in the same manner.

Accelerated Gaussian Elimination of $\mathbf{H}'$. Consider the first $(n - k)$ columns of $\mathbf{H}'$ and $\mathbf{H}$. It is evident that they share a large number of columns. This implies that if $\mathbf{H}$ is already in systematic form with respect to the first $n - k$ columns, the Gaussian elimination process in (3) can be expedited.

Specifically, assume $\mathbf{H}$ is in systematic form, and denote

$$l := \lfloor \frac{n - k}{\beta} \rfloor,$$

which represents the number of blocks contained in the first $(n - k)$ positions. After removing columns of $\mathbf{H}$ using the method described in Section 3.3, at least $l(\beta - \bar{u} - 1)$ of the first $(n - k)$ columns of $\mathbf{H}$ remain in the first $(n - k)$ columns of the resulting matrix. Consequently, we only need to execute

$$T_1 = (n - k)(n - k - l(\beta - \bar{u} - 1))(n_0 + n - k - l(\beta - \bar{u} - 1)) \quad (12)$$

field operations to transform the new matrix into systematic form, which is significantly less than the $(n - k)^2(n_0 + n - k)$ operations required in (3).

4 Complexity Analysis and Comparisons

This section presents the analysis of the complexity of Algorithm 1 as well as the complexity with respect to concrete RSD parameters. Note that we only present

the complexity results for large field sizes (e.g., $q = 2^{128}$) in this section since we propose more dedicated optimizations for binary RSD in [Section 5](#), and thus the corresponding complexity results will be discussed there.

4.1 Complexity Analysis

[Theorem 1](#) analyzes the complexity of [Algorithm 1](#) with the optimizations discussed in [Section 3.3](#) implemented. For clarity, we summarize the involved parameters as follows:

- $\bar{f}$, $\bar{u}$, and g satisfy $\bar{f} + g \leq h$, $\bar{u} + 1 \leq \beta - 1$, and $\bar{f} + g \leq h$. Here, $(\bar{f}, \bar{u})$ denotes the number of blocks and error-free positions defined in [Section 3.3](#), and g is the number of blocks containing coefficient variables defined in [Section 3.2](#).
- $m_1 = \bar{f} \binom{\beta - \bar{u} - 1}{2} + (h - g - \bar{f}) \binom{\beta - \bar{u}}{2}$ represents the number of quadratic equations in [\(6\)](#).
- $n_0 = k - \bar{f} - (h - g)\bar{u}$ represents the number of variables in $\mathbf{e}^{(2)}$ as defined in [\(4\)](#).
- $w = g\beta$ and $v = n_0 - w$ denote the number of coefficient variables and unknown variables, respectively, in [\(6\)](#).
- $z = v + 2g + gv + \binom{g}{2}$ represents the number of monomials after the partial enumeration in [Section 3.3](#). Additionally, z satisfies the condition $m_1 - \frac{v(v+1)}{2} \geq \frac{5z}{2}$ as stated in [\(9\)](#).
- $l = \lfloor \frac{n-k}{\beta} \rfloor$ represents the number of blocks in the first $(n - k)$ columns, as defined in [Section 3.3](#).

Theorem 1. *The $\text{RSD}(n, k, h)$ problem can be solved in time complexity*

$$T_{total} = P'_{\bar{f}, \bar{u}}^{-1} \cdot (T_1 + T_2 + T_{enum} \cdot T_3) \quad (13)$$

and space complexity $\mathcal{O}((n - k)n + m_1(n_0^2 + 3n_0)/2)$, where

$$P'_{\bar{f}, \bar{u}} = (1 - \frac{\bar{u}+1}{\beta})^{\bar{f}} (1 - \frac{\bar{u}}{\beta})^{(h-g-\bar{f})}$$

$$T_1 = (n - k)(n - k - l(\beta - \bar{u} - 1))(n_0 + n - k - l(\beta - \bar{u} - 1))$$

$$T_2 = \binom{n_0^2 + 3n_0}{2} \omega, \quad T_3 = \left(\frac{5z}{2}\right)^\omega + \frac{5z}{2}(v + 2w + wv + \binom{w}{2}), \quad T_{enum} = \beta^g.$$

and the $\omega = 2.8$ is the constant of Strassen's algorithm.

Proof. As discussed in [Section 3](#), if the positions selected in the first step are error-free, a solution to the RSD problem can be obtained. As a result, the algorithm will terminate after an average of $P'_{\bar{f}, \bar{u}}^{-1}$ iterations. The time complexity of each iteration is composed of the following four parts.

The first part is the complexity of performing Gaussian elimination on $\mathbf{H}'$, which equals T_1 as presented in [\(12\)](#).

The second part is the complexity of constructing the quadratic equations in [\(5\)](#). The construction of each quadratic equation (corresponding to a row of

$\mathbf{W}$) involves the multiplication of two polynomials, each with at most n_0 terms, costing n_0^2 field operations. Therefore, the time complexity for constructing $\mathbf{W}$ is $m_1 \cdot n_0^2$. However, as analyzed in Section 3.2, we only need to calculate $\frac{v(v+1)}{2} + \frac{5z}{2}$ equations (instead of m_1) to ensure the subsequent steps can be executed. Thus, the time complexity of this part is given by $T_{cons} = (\frac{v(v+1)}{2} + \frac{5z}{2})n_0^2 \leq \mathcal{O}(n_0^4)$ because $v < n_0$ and $g < w < n_0$.

The third part is the complexity of preprocessing the equation system, which involves performing partial Gaussian elimination of the matrix

$$\mathbf{W}' = [\mathbf{W}'_1 \parallel \mathbf{W}'_2 \parallel \mathbf{d}']$$

in (7). Note that $\mathbf{W}'_1 \in \mathbb{F}_q^{m_1 \times v(v+1)/2}$ is a matrix whose elements are in $\mathbb{F}_q$, $\mathbf{W}'_2$ is an $m_1 \times v$ matrix where each entry is an $\mathbb{F}_q$ -linear combination of $\tilde{e}_{v+1}, \dots, \tilde{e}_{v+w}$ and 1, and $\mathbf{d}'$ is a vector where each entry is a polynomial of $\tilde{e}_{v+1}, \dots, \tilde{e}_{v+w}$ with degree no more than 2. Therefore, each row operation of $\mathbf{W}'$ consumes $\frac{1}{2}v(v+1) + v(w+1) + \frac{1}{2}w(w+1) + w \approx \frac{1}{2}n_0^2$ field operations. We can assume the number of rows in $\mathbf{W}'$ is less than $\frac{1}{2}(n_0^2 + 3n_0)$, otherwise the algorithm is degenerated and we can solve the overdetermined polynomial system by performing Gaussian elimination on $\mathbf{W}'$. Then the complexity of partial Gaussian elimination on $\mathbf{W}'$ is bounded by the Gaussian elimination on a square matrix with number of rows $\frac{1}{2}(n_0^2 + 3n_0)$. Thus the time complexity of this part at most $T_2 = (\frac{n_0^2 + 3n_0}{2})\omega$, where the ω comes from the speedup of advanced technique for Gaussian elimination.

The fourth part involves the complexity of enumerating the nonzero positions of $\tilde{e}_{v+1}, \dots, \tilde{e}_{v+w}$ and solving the resulting polynomial system. As analyzed in Section 3.2, enumerating the nonzero positions requires $T_{enum} = \beta^g$ operations. For each enumeration, we substitute $\tilde{e}_{v+1}, \dots, \tilde{e}_{v+w}$ into (8) and solve the resulting system. Similarly, we can assume that the matrix $\mathbf{W}'^{(1)}_2$ in (8) has $\frac{5}{2}z$ rows by ignoring the extra rows. Thus, the time complexity for constructing the system is $\frac{5z}{2}(v + 2w + wv + \binom{w}{2})$, and the time complexity for solving the equation system is at most $(\frac{5z}{2})^\omega$. This results in a combined complexity of $T_3 = (\frac{5z}{2})^\omega + \frac{5z}{2}(v + 2w + wv + \binom{w}{2})$ for each substitution and solving process. Therefore, the overall complexity of this part is $T_{enum} \cdot T_3$.

Given that T_{cons} is much less than T_2 , we can deduce that the total time complexity is given by (13).

For the space complexity, the primary components are storing the matrix $\mathbf{H}$ for the RSD(n, k, h) problem, which requires $(n - k)n$ space, and storing the matrix $\mathbf{W}$ for constructing quadratic equations, which requires $m_1(n_0^2 + 3n_0)/2$ space. Thus, the total space complexity follows. $\square$

4.2 Time Complexity for Concrete RSD Parameters

Our selected RSD parameter sets align with those used in prior works, such as [31, 17, 26, 29], to facilitate a direct comparison. Additionally, the complexity is expressed in terms of the number of arithmetic operations over $\mathbb{F}_q$, consistent

Table 5. The time complexity for the number of field operations for large weight RSD over $\mathbb{F}_{2^{128}}$.

RSD(n, k, h)	AGB in [17]	ISD in [31]	Regular-ISD in [26]	RP in [29]	Ours	Parameters ($\bar{f}, \bar{u}, g$)
$(2^{22}, 64770, 4788)$	110	152	173	151	89.27	(1599,0,3)
$(2^{20}, 32771, 2467)$	131	155	173	148	114.27	(691,4,3)
$(2^{18}, 15336, 1312)$	133	153	160	147	119.03	(152,5,3)
$(2^{16}, 7391, 667)$	150	151	160	142	131.94	(620,6,3)
$(2^{14}, 3482, 338)$	150	150	153	139	133.15	(148,7,2)
$(2^{12}, 1589, 172)$	150	155	148	140	132.60	(21,7,2)
$(2^{10}, 652, 106)$	179	194	173	159	158.21	(18,5,2)

Table 6. The time complexity for the number of field operations for small weight RSD over $\mathbb{F}_{2^{128}}$.

RSD(n, k, h)	AGB in [17]	ISD in [31]	Regular-ISD in [26]	RP in [29]	Ours	Parameters ($\bar{f}, \bar{u}, g$)
$(2^{22}, 64770, 2735)$	104	108	126	105	86.51	(0,0,0)
$(2^{20}, 32771, 1419)$	102	107	122	103	83.91	(86,1,3)
$(2^{18}, 15336, 760)$	107	105	117	100	90.95	(532,5,3)
$(2^{16}, 7391, 389)$	111	103	113	97	96.39	(293,9,2)
$(2^{14}, 3482, 198)$	110	101	107	94	95.62	(38,11,2)
$(2^{12}, 1589, 98)$	107	100	101	89	93.08	(7,12,2)
$(2^{10}, 652, 57)$	111	111	106	98	97.61	(30,9,1)

with these works. The comparisons are organized into two categories based on error weight, as previously adopted: small error weight and large error weight, which are detailed in Table 5 and Table 6 respectively.

As previously mentioned, in this section we focus on comparisons over large fields for $q = 2^{128}$.

Comparison with the Algebraic Approach in [17]. As demonstrated in Table 5 and Table 6, our new hybrid algorithm outperforms the approach in [17] across all parameters listed. The gap can be as large as 21 bits when $(n, k, h) = (2^{10}, 652, 106)$. We attribute this advantage primarily to the suitability of our hybrid approach for solving the quadratic equations, which effectively leverages the regular structure when enumerating the variables, as detailed in Section 3.2.

Comparison with the ISD Algorithms in [31,26,29]. As indicated in Table 5 and Table 6, our algorithm demonstrates superior performance compared

with the ISD algorithms in [31,26] over all parameters. Compared with the results in [29], our algorithm outperforms in most given parameters. When the code rate $R = k/n$ is low, the advantage can be as substantial as approximately 63 bits for $(n, k, h) = (2^{22}, 64770, 4788)$. For parameters with higher code rates, the advantage decreases and our hybrid approach is slightly less effective than the ISD approaches in two cases, with a relatively small gap ranging from 1 to 4 bits. It is worth noting that lower code rates are often preferred in RSD-based constructions due to their association with reduced communication complexity, as detailed in [44,14,13].

Remark 3. For the RSD parameters in these tables, n/h may not be an integer. A common approach to address this is *truncation*, which assumes that the last $n - h\lfloor n/h \rfloor$ variables of $\mathbf{e}$ in $\text{RSD}(n, k, h)$ are always zero. This transforms $\text{RSD}(n, k, h)$ into an $\text{RSD}(h\lfloor n/h \rfloor, k + h\lfloor n/h \rfloor - n, h)$ instance by truncating the last $n - h\lfloor n/h \rfloor$ positions. The truncation method has been employed in [17,26,29]. The complexity results presented in Table 5 and Table 6 are based on truncation.

Alternatively, another approach is to set the block length as $\beta = \lfloor n/h \rfloor$ and leave the last $n - h\lfloor n/h \rfloor$ positions unmodified (without truncation). This method is adopted in [31]. We can also adapt our algorithm to exclude truncation and provide an additional comparison with [31] in Supplementary Material D.

4.3 Asymptotic Complexity

In this subsection, we analyze the asymptotic performance of our hybrid algorithm under various parameter settings. For simplicity, our analysis is restricted to the basic model, excluding the additional optimizations introduced in Section 3.3.

Parameters Leading to Polynomial Complexity. To begin with, the following theorem identifies the parameter region where the complexity is bounded by a polynomial in n .

Theorem 2. *Let $R = k/n$ and $\rho = h/n$. The total complexity T_{total} of our hybrid algorithm is at most a polynomial in n if $R^2\rho \leq n^{-1}$.*

Proof. We set $(f, u, g) = (0, 0, 0)$, resulting in $P_{f,u} = 1$ and $n_0 = k = Rn$. Therefore, we have $m_1 = \beta(\beta - 1)h/2 \approx \beta^2 h/2 = n/(2\rho)$. The number of variables equals $\binom{n_0}{2} + n_0 + n_0 \approx \frac{n_0^2}{2}$. Note that the quadratic equation system in (5) is overdetermined as long as $(n_0^2 + 3n_0)/2 \leq m_1$, i.e., $Rn \leq \sqrt{n/\rho}$ asymptotically, which is equivalent to

$$R^2\rho \leq n^{-1}. \quad (14)$$

In this case, we conclude that $T_{\text{total}} = T_1 + T_2$, which implies a polynomial complexity. $\square$

We note that [Theorem 2](#) applies to arbitrary field sizes, whereas previous works have only addressed the parameter region for polynomial complexity in the context of *binary* RSD [\[26,17\]](#). In [Section 5](#), we propose additional optimizations specifically for binary RSD and demonstrate refined results in the parameter region for polynomial complexity. Therefore, we defer the detailed discussion and comparison for binary RSD to [Section 5](#).

Asymptotic Complexity of the Hybrid Algorithm. The following theorem analyzes the asymptotic complexity of our hybrid algorithm for cases where the parameters do not satisfy the conditions specified in [Theorem 2](#).

Theorem 3. *Let $R = k/n$ and $\rho = h/n$. When $R^2\rho > n^{-1}$ and $n \rightarrow \infty$, the total complexity T_{total} of our hybrid algorithm satisfies the following: T_{total} is at most $\mathcal{O}\left(2^{\rho n \log_2\left(\frac{1}{1-R}\right)}\right)$.*

Proof. We set $g = 0$, $f = h$, and $u = R\beta$. This leads to $n_0 = k - fu = k - R\beta h = 0$, and $m_1 = h\binom{\beta-u}{2} > 0$, ensuring that the condition in [\(9\)](#) is always satisfied. For $g = 0$, we have $T_{\text{enum}} = \beta^g$ as a constant. Thus, the total complexity is at most $T_{\text{total}} = \mathcal{O}(P_{f,u}^{-1}) = \mathcal{O}\left((1 - \frac{u}{\beta})^{-f}\right) = \mathcal{O}\left(2^{\rho n \log_2\left(\frac{1}{1-R}\right)}\right)$. $\square$

5 The Hybrid Algorithm for Binary RSD

For binary RSD, [Algorithm 1](#) can be applied in the same manner. However, the binary case admits several optimizations and differences that lead to improved total complexity:

- (1) The regular property can be exploited to derive additional linear equations. These equations are then used to transform the RSD problem into an RSD' problem of smaller size, as demonstrated in [\[31,26,19\]](#).
- (2) In the construction of the quadratic equation system in [Section 3.1](#), quadratic monomials of the form x^2 do not appear. This reduces the size of the matrix $\mathbf{W}$ in [\(5\)](#).
- (3) During the partial enumeration of variables in [Section 3.2](#), enumerating positions is equivalent to enumerating all possible values. Consequently, enumerating positions transforms [\(8\)](#) into a linear system of equations, which can be solved via Gaussian elimination directly.

5.1 Differences for Binary RSD

We start by detailing the key differences between the binary case and the $\mathbb{F}_q$ case in [Section 3](#).

Transform to the RSD' Problem. Observe that the condition $w_H(\mathbf{e}_i) = 1$ in [Definition 1](#) corresponds to a parity equation for each block, given by $\sum_{j=1}^{\beta} e_{i,j} = 1$. These parity equations can be utilized to eliminate h columns of the matrix $\mathbf{H}$, reducing the $\text{RSD}(n, k, h)$ problem to the following $\text{RSD}'(n, k, h)$ problem in $n(n - k)$ bit operations.

Definition 2 (RSD' Problem). Suppose that n, k, h and $\beta = n/h$ are positive integers. Given a parity check matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k) \times (n-h)}$, a syndrome vector $\mathbf{s} \in \mathbb{F}_2^{n-k}$, then the $\text{RSD}'(n, k, h)$ aims to find a nonzero vector $\mathbf{e} = (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_h)$ such that $\mathbf{H}\mathbf{e} = \mathbf{s}$, $w_H(\mathbf{e}_i) \leq 1$ and $\mathbf{e}_i \in \mathbb{F}_2^{\beta-1}$ for $1 \leq i \leq h$.

It is evident that the solutions for the original $\text{RSD}(n, k, h)$ instance can then be retrieved from the solutions for the $\text{RSD}'(n, k, h)$ instance by utilizing the parity equation $\sum_{j=1}^{\beta} e_{i,j} = 1$ to recover the eliminated variable².

Guess Error-Free Positions. Same to that in [Algorithm 1](#), the parameters input to the algorithm satisfy $f, g, u \geq 0$, $f + g \leq h$, $u \leq \beta - 1$.

Although the block size for $\text{RSD}'(n, k, h)$ is $\beta - 1$ instead of β , the probability of guessing error-free positions remains unchanged in the RSD' problem. Specifically, we can assume that $\Pr(w_H(\mathbf{e}_i) = 0) = \frac{1}{\beta}$ and $\Pr(w_H(\mathbf{e}_i) = 1) = 1 - \frac{1}{\beta}$ for $1 \leq i \leq h$ in the RSD' problem. This assumption is achieved by uniformly selecting the h columns to be eliminated within each block during the transformation to the RSD' problem. Consequently, within one block, the probability that the u variables are all 0 is $(1 - \frac{u}{\beta-1}) \cdot \Pr(w_H(\mathbf{e}_i) = 1) + 1 \cdot \Pr(w_H(\mathbf{e}_i) = 0) = 1 - \frac{u}{\beta}$, which coincides with the probability derived in [Section 3.1](#).

Construct Quadratic Equations. The process of performing Gaussian elimination and constructing quadratic equations is identical to that described in [Section 3.1](#). The only difference lies in the quadratic equation $\mathbf{W}\mathbf{x} = \mathbf{d}$ in (5), where the matrix $\mathbf{W}$ now has size $m_0 \times \frac{n_0^2 + n_0}{2}$. Here, $m_0 = f \cdot \binom{\beta-u-1}{2} + (h-f) \cdot \binom{\beta-1}{2}$ and $n_0 = k - h - fu$. The number of rows m_0 changes because the block length in RSD' is $\beta - 1$ instead of β . The number of columns $(n_0^2 + n_0)/2$ is due to the absence of quadratic monomials $\{\tilde{e}_i^2\}_{1 \leq i \leq n_0}$.

Solve the Quadratic Equations. The process proceeds in the same manner as that in [Section 3.3](#), with the difference that after partial enumeration, (8) is transformed into a linear system and solved via Gaussian elimination directly. The key issue here remains ensuring that the matrix $\mathbf{W}_2^{(1)}$ in (8) has significantly more rows than columns, which guarantees: 1) The Gaussian elimination in (7)

² There is a possibility that the eliminated h columns correspond to all the 1's of $\mathbf{e}$ in the RSD instance, leading to a situation where such a solution cannot be recovered from the $\text{RSD}'(n, k, h)$ instance. This can be addressed by verifying whether the elimination falls into this category.

succeeds with high probability; 2) The resulting linear system is well-constrained. We note that $\mathbf{W}'_2^{(1)}$ is an $(m_1 - \frac{v(v-1)}{2}) \times v$ matrix in this case. Consequently, we require the input parameters to satisfy

$$m_1 - \frac{v(v-1)}{2} > \frac{5v}{2}, \quad (15)$$

for reasons analogous to those underlying condition (9).

5.2 Complexity Analysis for Binary RSD

The following theorem summarizes the complexity for the binary case. We note that the two techniques presented in Section 3.3 are also applicable to binary RSD and are therefore included in the analysis. Similar to Theorem 1, the analysis involves parameters $\bar{f}$, $\bar{u}$, and $g \geq 0$ satisfying $\bar{f} + g \leq h$, $\bar{u} + 1 \leq \beta - 1$, and $v \leq \sqrt{2m_1} - 2$, where $m_1 = \bar{f} \binom{\beta - \bar{u} - 2}{2} + (h - g - \bar{f}) \binom{\beta - \bar{u} - 1}{2}$ is the number of quadratic equations, $n_0 = k - h - \bar{f} - (h - g)\bar{u}$ is the total number of unknown variables, and $v = n_0 - g(\beta - 1)$ is the number of variables after the partial enumeration. Additionally, $w = g(\beta - 1)$ and $l = \lfloor \frac{n-k}{\beta-1} \rfloor$ are defined similarly.

Theorem 4. *The binary RSD(n, k, h) problem can be solved in time complexity*

$$T_{total} = P'_{\bar{f}, \bar{u}}^{-1} \cdot (T_1 + T_2 + T_{enum} \cdot T_3) \quad (16)$$

and space complexity $\mathcal{O}((n - k)n + m_1(n_0^2 + n_0)/2)$, where

$$P'_{\bar{f}, \bar{u}} = (1 - \frac{\bar{u}+1}{\beta})^{\bar{f}} (1 - \frac{\bar{u}}{\beta})^{(h-g-\bar{f})}$$

$$T_1 = (n - k)(n - k - l(\beta - \bar{u} - 2))(n_0 + n - k - l(\beta - \bar{u} - 2))$$

$$T_2 = \frac{v^4 n_0^2}{8 \log_2(v^2/2)}, \quad T_3 = \frac{5}{4}(2v^2 w + vw^2) + \frac{(5v/2)^3}{\log_2(5v/2)} \text{ and } T_{enum} = \beta^g.$$

Proof. The total complexity is derived in a manner similar to that of Theorem 1. The primary differences lie in the expressions for T_{cons} , T_2 , and T_3 , which are analyzed as follows.

For T_{cons} , similar to the analysis in Theorem 1, we only need to compute $v(v-1) + \frac{5v}{2} < \frac{(v+2)^2}{2}$ equations, as indicated by (15). Thus, the time complexity is given by $T_{cons} = \frac{1}{2}(v+2)^2 n_0^2 \approx \frac{1}{2}v^2 n_0^2$.

For T_2 , the number of rows in $\mathbf{W}'$ is at most $\frac{1}{2}(v+2)^2$ and the cost of each row operation is less than $n_0^2/2$. Based on the complexity of binary Gaussian elimination discussed in Section 2.4, the Gaussian elimination costs $(\frac{(v+2)^2}{2})^2 \frac{1}{\log_2((v+2)^2/2)}$ row operation. Consequently, the time complexity is given by $T_2 = \frac{1}{4} \cdot \frac{(v+2)^4}{\log_2((v+2)^2/2)} \cdot \frac{1}{2} n_0^2 \approx \frac{v^4 n_0^2}{8 \log_2(v^2/2)}$.

For T_3 , we assume that the matrix $\mathbf{W}'_2^{(1)}$ is a $\frac{5}{2}v \times v$ matrix. The time complexity for substituting variables is approximately $\frac{5v}{2}(vw + v + w + \binom{w}{2}) \approx \frac{5}{4}(2v^2 w + vw^2)$ and the time complexity for solving the linear equations is $(5v/2)^3 / \log_2(5v/2)$, resulting in $T_3 = \frac{5}{4}(2v^2 w + vw^2) + \frac{(5v/2)^3}{\log_2(5v/2)}$. $\square$

Table 7. The time complexity for large weight binary RSD.

RSD(n, k, h)	AGB in [17]	ISD in [31]	Regular-ISD in [26]	RP in [29]	Ours	Parameters ($\bar{f}, \bar{u}, g$)
($2^{22}, 64770, 4788$)	103	135	140	143	87.04	(0,0,3)
($2^{20}, 32771, 2467$)	125	135	138	139	105.44	(257,3,4)
($2^{18}, 15336, 1312$)	122	129	126	137	108.27	(1255,3,4)
($2^{16}, 7391, 667$)	139	125	126	130	119.67	(518,5,4)
($2^{14}, 3482, 338$)	138	122	118	125	118.53	(100,6,3)
($2^{12}, 1589, 172$)	135	121	109	120	113.69	(167,5,3)
($2^{10}, 652, 106$)	145	159	113	117	115.58	(20,4,8)

Table 8. The time complexity for small weight binary RSD.

RSD(n, k, h)	AGB in [17]	ISD in [31]	Regular-ISD in [26]	RP in [29]	Ours	Parameters ($\bar{f}, \bar{u}, g$)
($2^{22}, 64770, 2735$)	103	99	103	102	86.92	(0,0,3)
($2^{20}, 32771, 1419$)	98	97	98	100	81.09	(77,0,3)
($2^{18}, 15336, 760$)	104	92	93	97	86.74	(523,4,3)
($2^{16}, 7391, 389$)	108	87	90	92	90.90	(124,8,3)
($2^{14}, 3482, 198$)	106	83	84	88	88.97	(153,9,3)
($2^{12}, 1589, 98$)	103	80	78	84	82.16	(69,12,6)
($2^{10}, 652, 57$)	101	90	76	84	78.44	(36,8,6)

Time Complexity for Concrete Binary RSD Parameters. The comparison of the complexity for binary RSD is summarized in Table 7 and Table 8, corresponding to the ‘large error weight’ and ‘small error weight’ cases, respectively.

As demonstrated in the tables, our algorithm outperforms the algebraic approach in [17] across all listed parameters. Additionally, our algorithm outperforms the ISD algorithms in [31,26] when the code rate is low, but exhibits slightly reduced effectiveness at higher code rates.

5.3 Asymptotic Complexity Analysis for Binary RSD

We note that, due to the modifications introduced in Section 5.1, the asymptotic complexity of the binary case differs from that of the $\mathbb{F}_q$ case discussed in Section 4.3. Therefore, we provide a detailed asymptotic complexity analysis for binary RSD, along with a comparison to the asymptotic results presented in [26,17]. For simplicity, the analysis focuses on the basic model, excluding the additional optimizations proposed in Section 3.3.

Parameters Leading to Polynomial Complexity. We first identify the parameter region where the complexity is bounded by a polynomial in n .

Theorem 5. *Let $R = k/n$ and $\rho = h/n$. The total complexity T_{total} of our hybrid algorithm is at most a polynomial in n if one of the following conditions holds:*

- (a) $R \leq \rho$;
- (b) $R > \rho$ and $(R - \rho)^2 \rho \leq n^{-1}$.

Proof. (a) When $R \leq \rho$, we assign $(f, u) = (0, 0)$, resulting in $P_{f,u} = 1$ and $k - h - fu < 0$. This implies that the linear equation system in (1) has more equations than unknowns, leading to a time complexity of the hybrid algorithm given by $T_1 = (n - k)^2(n - h)$. Thus, T_{total} is at most a polynomial in n .

(b) When $R > \rho$, we set $(f, u, g) = (0, 0, 0)$, resulting in $P_{f,u} = 1$ and $n_0 = k - h = (R - \rho)n$. Therefore, we have $\sqrt{2m_1} = \sqrt{(\beta - 1)(\beta - 2)h} \approx \sqrt{\beta^2 h} = \sqrt{n/\rho}$. Note that the quadratic equation system in (5) is overdetermined as long as $n_0 \leq \sqrt{2m_1}$, i.e., $(R - \rho)n \leq \sqrt{n/\rho}$, which is equivalent to

$$(R - \rho)^2 \rho \leq n^{-1}. \quad (17)$$

In this case, we conclude that $T_{\text{total}} = T_1 + T_2$, which implies a polynomial complexity. $\square$

We note that the parameter region for polynomial complexity has also been discussed for the ISD method in [26] and the algebraic method in [17]. For the ISD method, [26] demonstrates that RSD with $R \leq \rho$ can be solved in polynomial time, coinciding with condition (a) in Theorem 5. For the algebraic method, [17] shows that the degree of regularity equals 2 (which corresponds to polynomial time complexity) when the RSD parameters satisfy $2b + a > 1$, where $R = n^{-b}$ and $\rho = n^{-a}$. It can be readily verified that this condition is a proper subset of condition (b) in Theorem 5.

Besides, the authors in [26] compare the polynomial complexity parameter regions of the RSD and SD problems over the binary field. They demonstrate that RSD exhibits a broader polynomial complexity parameter region when the code rate R and error rate ρ are restricted to constants. Our results extend this observation to cases where R and ρ are *sub-constant*. Specifically, for $R, \rho \rightarrow 0$ as $n \rightarrow \infty$, the polynomial complexity parameter region for ISD can be represented by $R\rho = \mathcal{O}(\log n/n)$. This is because the asymptotic complexity of the SD problem is given by $2^{\log_2(\frac{1}{1-R})\rho n} \approx 2^{R\rho n/\ln 2}$. On one hand, we show that within this parameter region, RSD can be solved in polynomial time. Specifically, by setting $f = h$ and $u = (R - \rho)\beta$ in our algorithm, the complexity of RSD can be expressed as $T_{\text{total}} = 2^{\rho n \log_2(\frac{1}{1-R+\rho})} \approx 2^{(R-\rho)\rho n/\ln 2}$. This is polynomial in n for the parameter region $R\rho = \mathcal{O}(\log n/n)$, since $(R - \rho)\rho n < R\rho n = \mathcal{O}(\log n)$. On the other hand, Theorem 5 demonstrates that RSD exhibits a broader polynomial complexity parameter region. For example, when $R = n^{-0.3}$ and $\rho = n^{-0.4}$, the complexity of SD is $2^{\mathcal{O}(n^{0.3})}$, while the complexity of RSD remains polynomial because it satisfies condition (b) of Theorem 5.

Asymptotic Complexity for Binary RSD. The following theorem provides the asymptotic complexity for binary RSD when the parameters do not satisfy the conditions outlined in [Theorem 5](#). Our analysis focuses on the following four cases, which align with those suggested in [\[17\]](#).

Theorem 6. *Let $R = k/n$ and $\rho = h/n$. When the conditions for polynomial complexity in [Theorem 5](#) are not satisfied and $n \rightarrow \infty$, the total complexity T_{total} of our hybrid algorithm satisfies the following:*

- if R and ρ are constants, then T_{total} is at most $\mathcal{O}\left(2^{\rho n \log_2\left(\frac{1}{1-R+\rho}\right)}\right)$.
- if R is a constant and $\rho = o(1)$, then T_{total} is at most $\mathcal{O}\left(2^{\rho n \log_2\left(\frac{1}{1-R}\right)}\right)$.
- if $R = o(1)$ and $\rho = o(R)$, then T_{total} is at most $\mathcal{O}\left(2^{\rho n \frac{R}{\ln 2}}\right)$.
- if $R = o(1)$ and $\rho = \lambda R$ with a constant $\lambda < 1$, then T_{total} is at most $\mathcal{O}\left(2^{\rho n \frac{(1-\lambda)R}{\ln 2}}\right)$.

Proof. (1) When R and ρ are constants, we set $g = 0$, $f = h$, and $u = (R - \rho)\beta$. This leads to $n_0 = k - h - fu = k - h - (R - \rho)\beta h = 0$, and $m_1 = h\left(\frac{\beta - u^{-1}}{2}\right) > 0$, ensuring that the condition that $v \leq \sqrt{2m_1} - 2$ is always satisfied. For $g = 0$, we have $T_{\text{enum}} = \beta^g$ as a constant. Thus, the total complexity is

$$T_{\text{total}} = \mathcal{O}(P_{f,u}^{-1}) = \mathcal{O}\left(\left(1 - \frac{u}{\beta}\right)^{-f}\right) = \mathcal{O}\left(2^{\rho n \log_2\left(\frac{1}{1-R+\rho}\right)}\right). \quad (18)$$

(2) When $\rho = o(1)$, we maintain the same choice of parameters g , f , and u . A similar deduction leads to a total complexity of $\mathcal{O}\left(2^{\rho n \log_2\left(\frac{1}{1-R+\rho}\right)}\right)$, which simplifies to $\mathcal{O}\left(2^{\rho n \log_2\left(\frac{1}{1-R}\right)}\right)$ since R is a constant.

(3) When $R = o(1)$ and $\rho = o(R)$, we again maintain the same choice of parameters g , f , and u . A similar deduction leads to a total complexity of $\mathcal{O}\left(2^{\rho n \log_2\left(\frac{1}{1-R+\rho}\right)}\right)$. Given that

$$\rho n \log_2\left(\frac{1}{1-R+\rho}\right) = \rho n \frac{\ln\left(1 + \frac{R-\rho}{1-R+\rho}\right)}{\ln 2} \sim \rho n \frac{1}{\ln 2} \frac{R-\rho}{1-R+\rho} \sim \rho n \frac{R}{\ln 2}, \quad (19)$$

we deduce that the total complexity is $\mathcal{O}\left(2^{\rho n \frac{R}{\ln 2}}\right)$.

(4) When $R = o(1)$ and $\rho = \lambda R$ with a constant $\lambda < 1$, we maintain the same choice of parameters g , f , and u . A similar deduction leads to a total complexity of $\mathcal{O}\left(2^{\rho n \log_2\left(\frac{1}{1-R+\rho}\right)}\right)$. Given that

$$\rho n \log_2\left(\frac{1}{1-R+\rho}\right) = \rho n \log_2\left(\frac{1}{1+(\lambda-1)R}\right) \sim \rho n \frac{(1-\lambda)R}{\ln 2}, \quad (20)$$

we conclude that the total complexity is $\mathcal{O}\left(2^{\rho n \frac{(1-\lambda)R}{\ln 2}}\right)$. $\square$

Next, we provide a comparison of the asymptotic complexity between the algebraic method in [17] and our hybrid method. We note that the latest ePrint version of [17] updates results for the asymptotic complexity at constant code rate and noise rate. Therefore, we take these revisions in comparison. As analyzed in [17], the asymptotic complexity of the algebraic algorithm is based on Gröbner basis computations [38], and is given by

$$T_{\text{AGB}} = M_{d_{\text{reg}}}^2, \quad \text{where} \quad M_{d_{\text{reg}}} = d_{\text{reg}} \cdot \binom{h}{d_{\text{reg}}} (\beta - 1)^{d_{\text{reg}}} \quad (21)$$

represents the number of monomials up to degree d_{reg} . It is shown in [17] that, as $n \rightarrow \infty$, the degree of regularity d_{reg} for binary RSD asymptotically follows:

1. For constant code rate R and constant noise rate ρ ,

$$d_{\text{reg}}(u) \sim \frac{(\sqrt{1 - \rho(u+1)} - \sqrt{1 - R})^2}{1 - (u+2)\rho} h,$$

where u is the number of variables to be guessed in each block. In the following, we set $\kappa_{R,\rho,u} := \frac{(\sqrt{1 - \rho(u+1)} - \sqrt{1 - R})^2}{1 - (u+2)\rho}$.

2. For constant code rate R and noise rate $\rho = o(1)$,

$$d_{\text{reg}} \sim n\rho \left(2 - R - 2\sqrt{1 - R}\right).$$

3. For $R = o(1)$ and $\rho = o(1)$, where $\rho = o(R)$,

$$d_{\text{reg}} + 1 \sim \frac{n\rho R^2}{4}.$$

4. For $R = o(1)$ and $\rho = \lambda R$ with a constant $\lambda < 1$,

$$d_{\text{reg}} + 1 \sim \frac{(1 - \lambda)^2 n\rho R^2}{4}.$$

For simplicity, we express the asymptotic complexity in logarithmic form (denoted as log-complexity). Utilizing the fact that $\log_2 \left(\binom{h}{d_{\text{reg}}} \right) \sim h \cdot \mathbb{H}(d_{\text{reg}}/h)$, where $\mathbb{H}(x) = -x \log_2 x - (1 - x) \log_2 (1 - x)$ represents the binary entropy function, we present the comparison as follows:

1. *Constant code rate R and error rate ρ .* The log-complexity of our algorithm is $\rho n \log_2 \frac{1}{1 - R + \rho}$, while the log-complexity of the algebraic approach is

$$\min_{0 \leq u < R/\rho - 1} [2 \log_2 (1/\rho - u - 1) \kappa_{R,\rho,u} \rho + 2\mathbb{H}(\kappa_{R,\rho,u}) \rho - \rho \log_2 (1 - u\rho)] n \quad (22)$$

When $u \rightarrow R/\rho - 1$, the log-complexity of algebraic approach is $-\rho \log_2 (1 - R + \rho) n$, which equals that of our algorithm. However, through minimization, the algebraic approach can achieve better efficiency.

2. *Constant code rate R and error rate $\rho = o(1)$.* The log-complexity of our algorithm is $\rho n \log_2(\frac{1}{1-R})$, whereas the log-complexity of the algebraic approach is

$$2n\rho(2 - R - 2\sqrt{1 - R}) \log_2 \frac{1}{\rho} + 2n\rho \cdot \mathsf{H}(2 - R - 2\sqrt{1 - R}). \quad (23)$$

For fixed R and $\rho = o(1)$, the term $\log_2 \frac{1}{\rho}$ grows very large, making the asymptotic log-complexity of [17] significantly higher than ours.

3. *Code rate $R = o(1)$ and error rate $\rho = o(R)$.* The asymptotic log-complexity of our algorithm is $\rho n \frac{R}{\ln 2}$, while the log-complexity of the algebraic approach is

$$2n\rho \frac{R^2}{4} \log_2 \frac{1}{\rho} + 2n\rho \cdot \mathsf{H}\left(\frac{R^2}{4}\right). \quad (24)$$

The first term, $2n\rho \frac{R^2}{4} \log_2 \frac{1}{\rho}$, dominates asymptotically. Thus, for $\frac{R}{\ln 2} < \frac{R^2}{2} \log_2 \frac{1}{\rho}$, we can deduce that the asymptotic complexity of our algorithm is lower than that of the algebraic method. This condition simplifies to $\rho < 2^{\frac{-2}{R \ln 2}}$.

4. *Code rate $R = o(1)$ and $\rho = \lambda R$ with a constant $\lambda < 1$.* The log-complexity of our algorithm is $\rho n \frac{(1-\lambda)R}{\ln 2}$, while the log-complexity of the algebraic approach is

$$2n\rho \frac{(1-\lambda)^2 R^2}{4} \log_2 \frac{1}{\rho} + 2n\rho \cdot \mathsf{H}\left(\frac{(1-\lambda)^2 R^2}{4}\right). \quad (25)$$

In this case, the asymptotic complexity of the algebraic approach is lower than that of our algorithm.

We summarize the comparison of asymptotic complexities in [Table 9](#).

Table 9. Superiority regions of our algorithm and the algebraic method in [17] under different code rates R and noise levels ρ . Here, $\emptyset$ denotes the empty set, and $\forall R, \rho$ refers to all possible cases under the given condition.

	Superiority Regions of Our Hybrid Algorithm	Superiority Regions of the Algebraic Method in [17]
constant R, ρ	$\emptyset$	$\forall R, \rho$
constant R and $\rho = o(1)$	$\forall R, \rho$	$\emptyset$
$R = o(1)$ and $\rho = o(R)$	$\rho < 2^{\frac{-2}{R \ln 2}}$	$\rho > 2^{\frac{-2}{R \ln 2}}$
$R = o(1)$ and $\rho = \lambda R$ with constant $\lambda < 1$	$\emptyset$	$\forall R, \rho$

Table 10. The estimated security level for Wolverine using our hybrid algorithm, the algebraic algorithm [17], and the ISD algorithms [26,29,31], where the field size $q = 2^{61} - 1$.

	Wolverine <i>one-time setup</i>	Wolverine <i>extend execution</i>
RSD(n, k, h)	(642048, 19870, 2508)	(10805248, 589760, 1319)
Target Security Level	128	128
Our Hybrid Algorithm	110.78	149.48
Algebraic Method in [17]	126	165
Regular-ISD in [26]	172	177
RP in [29]	147	155
ISD in [31]	154	160

6 Application to RSD-Based Protocols

In this section, we demonstrate the effectiveness of our hybrid algorithm by applying it to concrete protocols Wolverine [42] and Ferret [44].

Application to Wolverine. Wolverine is a protocol for constant-round interactive ZK proofs, designed to achieve asymptotically optimal memory usage and lower communication costs [42]. Its security relies on the RSD problem over $\mathbb{F}_q$ with $q = 2^{61} - 1$, and it proposes a one-time setup parameter set and an extend execution parameter set, both targeting 128-security level.

We apply our hybrid algorithm from Section 3, along with the algebraic method from [17] and the ISD algorithms from [31,26,29], to evaluate Wolverine’s security. The time complexities are summarized in Table 10, where the parameters $(\bar{f}, \bar{u}, g)$ for the hybrid algorithm are set to (1444, 2, 3) for the one-time setup case and to (588, 209, 3) for the extended execution case. According to the table, our algorithm outperforms both the algebraic method and the ISD algorithms, indicating that the security level of the one-time setup parameter set is at most 110.61, which falls below the targeted 128-security level.

Application to Ferret. Ferret is a correlated oblivious transfer protocol introduced in [44]. It includes Ferret-Reg, which comprises a one-time setup corresponding to a binary RSD(609728, 36288, 1269) instance, and a main iteration corresponding to a binary RSD(10805248, 589760, 1319) instance. Both parameter sets aim to achieve a 128-bit security level.

We apply our hybrid algorithm from Section 5, along with the algebraic method from [17] and the ISD algorithms from [31,26,29], to evaluate the security of Ferret. The time complexities are summarized in Table 11, where the parameters $(\bar{f}, \bar{u}, g)$ for the hybrid algorithm are set to (1220, 12, 3) for the one-time

Table 11. The estimated security level for Ferret using our hybrid algorithm, the algebraic algorithm [17], and the ISD algorithms [26,29,31], where the field size $q = 2$.

	Ferret-Reg one-time setup	Ferret-Reg main iteration
RSD(n, k, h)	(609728, 36288, 1269)	(10805248, 589760, 1319)
Target Security	128	128
Our Hybrid Algorithm	126.59	149.90
Regular-ISD in [26]	140	157
Algebraic Method in [17]	143	164
RP in [29]	144	155
ISD in [31]	138	152

setup case and (580, 208, 3) for the main iteration case. According to Table 11, our algorithm outperforms both the algebraic method and the ISD algorithms for these parameter sets. Furthermore, for the one-time setup parameter set, our algorithm indicates that the security level is slightly below the targeted 128-bit security level.

7 Conclusion

We propose a new hybrid algorithm for the RSD problem over arbitrary fields and adapt it to binary field with optimizations. Our algorithm consists of two main components: constructing quadratic equations via Gaussian elimination and solving these equations using a hybrid algebraic method. We demonstrate that our algorithm exhibits lower time complexity across a wide range of parameters compared to the previous algebraic approach in [17]. When compared to ISD algorithms, our method proves particularly efficient over large fields and low code rates. We have conducted a comprehensive analysis of the asymptotic complexity for varying code and error rates. For binary RSD, our algorithm reveals a broader polynomial-time complexity region outperforming both ISD and algebraic approaches. Finally, we detail in the performance of our algorithm on concrete protocols Wolverine and Ferret.

Several issues remain open for future work. Firstly, our hybrid technique, which involves guessing error-free positions, raises the question of whether allowing some errors outside the information set could enable the use of the meet-in-the-middle technique to facilitate solving quadratic equations. Secondly, the procedure of our hybrid algorithm can be summarized as ‘guess and solve’ iterations, which involve guessing some variables, eliminating some variables, enumerating additional coefficients and solving the unknowns. It remains unclear whether a more complex ‘guess and solve’ procedure would result in lower complexity.

Acknowledgments

This work is supported by the National Cryptologic Science Fund of China (2025NCSF02041), the National Key R&D Program of China (2020YFA0309705), the National Natural Science Foundation of China (12222113, 12441105), the Scientific Research Innovation Capability Support Project for Young Faculty (ZYGXQNJSKYCXNLZCXM-P4), the Key R&D Program of Shandong Province (2024ZLGX05), and Tsinghua University Dushi Program.

References

1. Abram, D., Scholl, P.: Low-communication multiparty triple generation for SPDZ from Ring-LPN. In: Hanaoka, G., Shikata, J., Watanabe, Y. (eds.) *Public-Key Cryptography – PKC 2022*. pp. 221–251. Springer International Publishing (2022)
2. Al Jabri, A.: A statistical decoding algorithm for general linear block codes. In: Honary, B. (ed.) *8th IMA International Conference on Cryptography and Coding*. *Lecture Notes in Computer Science*, vol. 2260, pp. 1–8. Springer, Heidelberg, Germany, Cirencester, UK (Dec 17–19, 2001)
3. Andr n, D., Hellstr m, L., Markstr m, K.: On the complexity of matrix reduction over finite fields. *Advances in applied mathematics* **39**(4), 428–452 (2007)
4. Ant Group: SecretFlow: A unified trusted privacy-preserving computing framework. <https://github.com/secretflow> (2022)
5. Augot, D., Finiasz, M., Sendrier, N.: A family of fast syndrome based cryptographic hash functions. In: Dawson, E., Vaudenay, S. (eds.) *Progress in Cryptology - Mycrypt 2005, First International Conference on Cryptology in Malaysia, Kuala Lumpur, Malaysia, September 28-30, 2005, Proceedings*. *Lecture Notes in Computer Science*, vol. 3715, pp. 64–83. Springer (2005). https://doi.org/10.1007/11554868_6, https://doi.org/10.1007/11554868_6
6. Baum, C., Malozemoff, A.J., Rosen, M.B., Scholl, P.: Mac’n’cheese: Zero-knowledge proofs for boolean and arithmetic circuits with nested disjunctions. In: Malkin, T., Peikert, C. (eds.) *Advances in Cryptology – CRYPTO 2021, Part IV*. *Lecture Notes in Computer Science*, vol. 12828, pp. 92–122. Springer, Heidelberg, Germany, Virtual Event (Aug 16–20, 2021). https://doi.org/10.1007/978-3-030-84259-8_4
7. Becker, A., Joux, A., May, A., Meurer, A.: Decoding random binary linear codes in $2^{n/20}$: How $1 + 1 = 0$ improves information set decoding. In: Pointcheval, D., Johansson, T. (eds.) *Advances in Cryptology – EUROCRYPT 2012*. *Lecture Notes in Computer Science*, vol. 7237, pp. 520–536. Springer, Heidelberg, Germany, Cambridge, UK (Apr 15–19, 2012). https://doi.org/10.1007/978-3-642-29011-4_31
8. Berlekamp, E.R., McEliece, R.J., van Tilborg, H.C.A.: On the inherent intractability of certain coding problems (corresp.). *IEEE Trans. Inf. Theory* **24**(3), 384–386 (1978). <https://doi.org/10.1109/TIT.1978.1055873>, <https://doi.org/10.1109/TIT.1978.1055873>
9. Bombar, M., Bui, D., Couteau, G., Couvreur, A., Ducros, C., Servan-Schreiber, S.: FOLEAGE: $\mathbb{F}_4$ OLE-based multi-party computation for boolean circuits. *Cryptology ePrint Archive*, Paper 2024/429 (2024), <https://eprint.iacr.org/2024/429>
10. Bombar, M., Couteau, G., Couvreur, A., Ducros, C.: Correlated pseudorandomness from the hardness of quasi-abelian decoding. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA,*

- August 20-24, 2023, Proceedings, Part IV. Lecture Notes in Computer Science, vol. 14084, pp. 567–601. Springer (2023). https://doi.org/10.1007/978-3-031-38551-3_18, https://doi.org/10.1007/978-3-031-38551-3_18
11. Bouillaguet, C., Chen, H.C., Cheng, C.M., Chou, T., Niederhagen, R., Shamir, A., Yang, B.Y.: Fast exhaustive search for polynomial systems in. In: International Workshop on Cryptographic Hardware and Embedded Systems. pp. 203–218. Springer (2010)
 12. Bouillaguet, C., Delaplace, C., Trimoska, M.: A simple deterministic algorithm for systems of quadratic polynomials over $\mathbb{F}_2$. In: Symposium on Simplicity in Algorithms (SOSA). pp. 285–296. SIAM (2022)
 13. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y.: Compressing vector OLE. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. pp. 896–912 (2018)
 14. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Resch, N., Scholl, P.: Correlated pseudorandomness from expand-accumulate codes. In: Dodis, Y., Shrimpton, T. (eds.) Advances in Cryptology – CRYPTO 2022, Part II. Lecture Notes in Computer Science, vol. 13508, pp. 603–633. Springer, Heidelberg, Germany, Santa Barbara, CA, USA (Aug 15–18, 2022). https://doi.org/10.1007/978-3-031-15979-4_21
 15. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Scholl, P.: Efficient pseudorandom correlation generators: Silent OT extension and more. In: Boldyreva, A., Micciancio, D. (eds.) Advances in Cryptology – CRYPTO 2019, Part III. Lecture Notes in Computer Science, vol. 11694, pp. 489–518. Springer, Heidelberg, Germany, Santa Barbara, CA, USA (Aug 18–22, 2019). https://doi.org/10.1007/978-3-030-26954-8_16
 16. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Scholl, P.: Efficient pseudorandom correlation generators from ring-LPN. In: Advances in Cryptology–CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part II 40. pp. 387–416. Springer (2020)
 17. Briaud, P., Øygarden, M.: A new algebraic approach to the regular syndrome decoding problem and implications for PCG constructions. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 391–422. Springer (2023)
 18. Bui, D., Carozza, E., Couteau, G., Goudarzi, D., Joux, A.: Faster signatures from MPC-in-the-head. Cryptology ePrint Archive, Paper 2024/252 (2024), <https://eprint.iacr.org/2024/252>
 19. Carozza, E., Couteau, G., Joux, A.: Short signatures from regular syndrome decoding in the head. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 532–563. Springer (2023)
 20. Courtois, N., Klimov, A., Patarin, J., Shamir, A.: Efficient algorithms for solving overdefined systems of multivariate polynomial equations. In: International Conference on the Theory and Applications of Cryptographic Techniques. pp. 392–407. Springer (2000)
 21. Couteau, G., Rindal, P., Raghuraman, S.: Silver: Silent VOLE and oblivious transfer from hardness of decoding structured LDPC codes. In: Malkin, T., Peikert, C. (eds.) Advances in Cryptology – CRYPTO 2021, Part III. Lecture Notes in Computer Science, vol. 12827, pp. 502–534. Springer, Heidelberg, Germany, Virtual Event (Aug 16–20, 2021). https://doi.org/10.1007/978-3-030-84252-9_17

22. Cui, H., Liu, H., Yan, D., Yang, K., Yu, Y., Zhang, K.: Resolved: Shorter signatures from regular syndrome decoding and VOLE-in-the-head. *Cryptology ePrint Archive* (2024)
23. Dinur, I.: Cryptanalytic applications of the polynomial method for solving multivariate equation systems over $\text{gf}(2)$. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. pp. 374–403. Springer (2021)
24. Dumer, I.: On minimum distance decoding of linear codes. In: *Proc. 5th Joint Soviet-Swedish Int. Workshop Inform. Theory*. pp. 50–52 (1991)
25. Esser, A., Kübler, R., May, A.: LPN decoded. In: Katz, J., Shacham, H. (eds.) *Advances in Cryptology – CRYPTO 2017, Part II. Lecture Notes in Computer Science*, vol. 10402, pp. 486–514. Springer, Heidelberg, Germany, Santa Barbara, CA, USA (Aug 20–24, 2017). https://doi.org/10.1007/978-3-319-63715-0_17
26. Esser, A., Santini, P.: Not just regular decoding: Asymptotics and improvements of regular syndrome decoding attacks. *Cryptology ePrint Archive* (2023)
27. Hazay, C., Orsini, E., Scholl, P., Soria-Vazquez, E.: Tinykeys: A new approach to efficient multi-party computation. In: *Annual International Cryptology Conference*. pp. 3–33. Springer (2018)
28. Joux, A., Vitse, V.: A crossbred algorithm for solving boolean polynomial systems. In: Kaczorowski, J., Pieprzyk, J., Pomykala, J. (eds.) *Number-Theoretic Methods in Cryptology - First International Conference, NuTMiC 2017, Warsaw, Poland, September 11-13, 2017, Revised Selected Papers. Lecture Notes in Computer Science*, vol. 10737, pp. 3–21. Springer (2017). https://doi.org/10.1007/978-3-319-76620-1_1, https://doi.org/10.1007/978-3-319-76620-1_1
29. Kim, J., Lee, C.: Reduce and Prange: Revisiting Prange’s information set decoding for LPN and RSD. *Cryptology ePrint Archive* (2024)
30. Lee, P.J., Brickell, E.F.: An observation on the security of mceliece’s public-key cryptosystem. In: *Workshop on the Theory and Application of Cryptographic Techniques*. pp. 275–280. Springer (1988)
31. Liu, H., Wang, X., Yang, K., Yu, Y.: The hardness of LPN over any integer ring and field for pcg applications. *Cryptology ePrint Archive* (2022)
32. May, A., Meurer, A., Thomae, E.: Decoding random linear codes in $\tilde{O}(2^{0.054n})$. In: Lee, D.H., Wang, X. (eds.) *Advances in Cryptology – ASIACRYPT 2011. Lecture Notes in Computer Science*, vol. 7073, pp. 107–124. Springer, Heidelberg, Germany, Seoul, South Korea (Dec 4–8, 2011). https://doi.org/10.1007/978-3-642-25385-0_6
33. Meta: Private Computation Framework 2.0. <https://github.com/facebookresearch/fbpcf> (2022)
34. Prange, E.: The use of information sets in decoding cyclic codes. *IRE Transactions on Information Theory* **8**(5), 5–9 (1962)
35. Rachuri, R., Scholl, P.: Le mans: Dynamic and fluid MPC for dishonest majority. In: Dodis, Y., Shrimpton, T. (eds.) *Advances in Cryptology – CRYPTO 2022, Part I. Lecture Notes in Computer Science*, vol. 13507, pp. 719–749. Springer, Heidelberg, Germany, Santa Barbara, CA, USA (Aug 15–18, 2022). https://doi.org/10.1007/978-3-031-15802-5_25
36. Raghuraman, S., Rindal, P., Tanguy, T.: Expand-convolute codes for pseudorandom correlation generators from LPN. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology – CRYPTO 2023*. pp. 602–632. Springer Nature Switzerland (2023)
37. Schoppmann, P., Gascón, A., Reichert, L., Raykova, M.: Distributed vector-OLE: Improved constructions and implementation. In: *Proceedings of the 2019 ACM*

- SIGSAC Conference on Computer and Communications Security. pp. 1055–1072 (2019)
38. Semaev, I., Tenti, A.: Probabilistic analysis on Macaulay matrices over finite fields and complexity of constructing Gröbner bases. *Journal of Algebra* **565**, 651–674 (2021)
 39. Stern, J.: A method for finding codewords of small weight. In: *Coding Theory and Applications: 3rd International Colloquium Toulon, France, November 2–4, 1988 Proceedings 3*. pp. 106–113. Springer (1989)
 40. Wang, X., Malozemoff, A.J., Katz, J.: Emp-toolkit: Efficient multiparty computation toolkit. <https://github.com/emp-toolkit> (2016)
 41. Weger, V., Gassner, N., Rosenthal, J.: A survey on code-based cryptography. *CoRR abs/2201.07119* (2022), <https://arxiv.org/abs/2201.07119>
 42. Weng, C., Yang, K., Katz, J., Wang, X.: Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In: *2021 IEEE Symposium on Security and Privacy*. pp. 1074–1091. IEEE Computer Society Press, San Francisco, CA, USA (May 24–27, 2021). <https://doi.org/10.1109/SP40001.2021.00056>
 43. Williams, R.R.: The polynomial method in circuit complexity applied to algorithm design (invited talk). In: *34th International Conference on Foundation of Software Technology and Theoretical Computer Science (FSTTCS 2014)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik (2014)
 44. Yang, K., Weng, C., Lan, X., Zhang, J., Wang, X.: Ferret: Fast extension for correlated ot with small communication. In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. pp. 1607–1626 (2020)

Supplementary Material

A Parameters of the ISD in [31] and [26]

We provide an analysis of the parameters of the ISD algorithms utilized in [31,26] over $\mathbb{F}_2$. We are particularly interested in the BJMM ISD in [31,26] as it typically exhibits the best time complexity among all ISD variants.

To begin with, we briefly recall the BJMM parameters. According to [7], BJMM initializes parameters l , p , ϵ_1 , ϵ_2 , r_1 , and r_2 , and constructs a tree of lists of vectors to enumerate. Here, $p_2/2$, p_2 , p_1 , and p denote the weights of vectors in layers 3, 2, 1, and 0 of the tree, respectively, where $p_1 = \epsilon_1 + p/2$ and $p_2 = \epsilon_2 + p_1/2$. The size of lists in each layer can be represented as

$$S_3 = \binom{(k+l)/2}{p_2/2}, S_2 = \frac{S_3^2}{2^{r_2}}, S_1 = \frac{S_2^2}{2^{r_1}}, \quad (26)$$

where $r_1 \approx \log_2\left(\binom{p}{\frac{p}{2}}\binom{k+l-p}{\epsilon_1}\right)$ and $r_2 \approx \log_2\left(\binom{p_1}{p_1/2}\binom{k+l-p_1}{\epsilon_2}\right)$.

For concrete RSD parameters, we present the parameters chosen in [31] and in [26] in Table 12. For simplicity, we only present the parameters $(p, p_1, p_2, p_2/2)$. According to the tables, we find that $(p, p_1, p_2, p_2/2)$ are small constants under most of the parameters. Specifically, we find $p_2/2 = 1$ in most cases, which makes the size of the list in layer 3, $S_3 = (k+l)/2$, a linear term of n . Moreover, we notice that sometimes the best complexity of regular-ISD in [26] is obtained under enumeration-based regular-ISD, a variant of Stern-Dumer rather than BJMM. In this case, the meet-in-the-middle technique is even more degraded.

B Comparison for AGB, Plain BDT and Ours

To clarify the source of improvement in our algorithm, we provide a comparison between AGB from [17], plain BDT, and our hybrid algorithm. The plain BDT is to replace the XL subroutine of AGB with the BDT algorithm with back substitution. We restrict the comparison to the case of $\mathbb{F}_2$ with some fixed (f, u) guess. In $\mathbb{F}_q$, the cost of guessing a single variable in plain BDT is q , which makes the complexity of plain BDT prohibitively high for large fields such as $\mathbb{F}_{2^{128}}$.

In Table 13, we observe that plain BDT is less effective than AGB in most cases, indicating that applying plain BDT with back substitution does not usually yield improvement over AGB. Consequently, the improvement in complexity primarily stems from our method of leveraging the regular structure to reduce enumeration space in BDT/Crossbred.

C Implementation

We present experiments with concrete parameters to validate the effectiveness of our algorithm over binary field. Specifically, we aim to confirm the following:

Table 12. The selection of $(p, p_1, p_2, p_2/2)$ in two BJMM-like algorithms.

RSD(n, k, h)	Regular-ISD in [26] $(p, p_1, p_2, p_2/2)$	BJMM in [31] $(p, p_1, p_2, p_2/2)$
$(2^{22}, 64770, 4788)$	(8, 4, 2, 1)	(6, 4, 2, 1)
$(2^{20}, 32771, 2467)$	(8, 4, 2, 1)	(6, 4, 2, 1)
$(2^{18}, 15336, 1312)$	(8, 4, 2, 1)	(6, 4, 2, 1)
$(2^{16}, 7391, 667)$	(8, 4, 2, 1)	(6, 4, 2, 1)
$(2^{14}, 3482, 338)$	(8, 4, 2, 1)	(10, 8, 4, 2)
$(2^{12}, 1589, 172)$	(16, 12, 6, 3)	(16, 12, 6, 3)
$(2^{10}, 652, 106)$	(24, 16, 8, 4)	(8, 4, 2, 1)
$(2^{22}, 64770, 2735)$	(8, 4, 2, 1)	(6, 4, 2, 1)
$(2^{20}, 32771, 1419)$	(8, 4, 2, 1)	(6, 4, 2, 1)
$(2^{18}, 15336, 760)$	(8, 4, 2, 1)	(6, 4, 2, 1)
$(2^{16}, 7391, 389)$	(8, 4, 2, 1)	(6, 4, 2, 1)
$(2^{14}, 3482, 198)$	(8, 4, 2, 1)	(6, 4, 2, 1)
$(2^{12}, 1589, 98)$	(8, 4, 2, 1)	(6, 4, 2, 1)
$(2^{10}, 652, 57)$	(16, 12, 6, 3)	(8, 4, 2, 1)

1) the Gaussian elimination of the matrix $\mathbf{H}'$ in Section 3.1 succeeds with at least a constant probability; 2) the partial Gaussian elimination of the matrix $\mathbf{W}'$ in Section 3.2 is successful; and 3) our algorithm works for specific RSD instances with smaller parameters. Under the assumption of randomness and independence, the success rate of Gaussian elimination for the same matrix over $\mathbb{F}_q$ where $q > 2$ is higher than that over $\mathbb{F}_2$. Therefore, we conduct the validation experiments exclusively in $\mathbb{F}_2$ and similar conclusions can be extended to larger fields.

The code for these experiments is available at <https://anonymous.4open.science/r/HybridRSD>.

C.1 Gaussian Elimination of the Matrix $\mathbf{H}'$

In this experiment, we perform Gaussian elimination on $\mathbf{H}'$ for RSD instances corresponding to both random linear codes and local linear codes³. To ensure the parameters align with the real RSD parameters in Table 7, we set (n, k, h) such that $k = 0.1n$, $h = 0.1k$, and n ranges between 500 and 7000. The selection of (f, u, g) has minimal impact on the results of Gaussian elimination, and we set them to $(f, u, g) = (2, 10, 1)$ for $n \geq 2000$ and $(f, u, g) = (2, 10, 0)$ for $n < 2000$. Additionally, for local linear codes, the column weight d is fixed at 15.

³ Local linear codes are frequently preferred by protocols over random linear codes due to their linear encoding time [44,13,37]. Specifically, the generator $\mathbf{G}_{k \times n}$ of local linear code has d nonzeros per column.

Table 13. The comparison for AGB, plain BDT and our algorithm in $\mathbb{F}_2$.

$\text{RSD}(n, k, h)$	(f, u)	AGB in [17]	plain BDT	Ours
$(2^{10}, 652, 106)$	(99, 4)	154	149	121
	(75, 5)	159	169	132
	(63, 6)	171	173	141
$(2^{12}, 1589, 172)$	(165, 6)	146	184	115
	(150, 7)	152	131	117
	(125, 8)	151	174	119
$(2^{14}, 3482, 338)$	(302, 7)	158	234	119
	(290, 8)	141	125	124
	(265, 9)	144	128	126
$(2^{16}, 7391, 667)$	(639, 6)	162	479	120
	(540, 7)	161	532	125
	(500, 8)	164	319	121
$(2^{10}, 652, 57)$	(53, 9)	113	87	84
	(50, 10)	105	90	89
	(40, 11)	117	118	92
$(2^{12}, 1589, 98)$	(93, 11)	116	180	85
	(75, 12)	112	291	96
	(73, 13)	115	245	93
$(2^{14}, 3482, 198)$	(191, 10)	125	308	89
	(180, 11)	127	243	90
	(150, 12)	124	412	97
$(2^{16}, 7391, 389)$	(357, 9)	132	563	91
	(340, 10)	134	382	92
	(280, 11)	131	692	93

We perform Gaussian elimination between 2000 and 20000 times for each parameter set, with the success rates displayed in Fig. 1. It is theoretically known that a random binary matrix is non-singular with probability $c \approx 0.289$ as $n \rightarrow \infty$. As shown in the figure, the success rate for random codes decreases sharply for small parameters and approaches the theoretical constant of 0.289 as n increases. On the other hand, the success rate for local linear codes initially decreases and then oscillates around 0.289 as n grows.

Moreover, in our experiments, we observe that Gaussian eliminations typically fail when only one or two rows remain. In such cases, the proposed algorithm can continue to operate by discarding the corresponding linear equations. Consequently, the number of unknown variables in (5) increases by one or two. This marginal increase has little impact on the overall complexity.

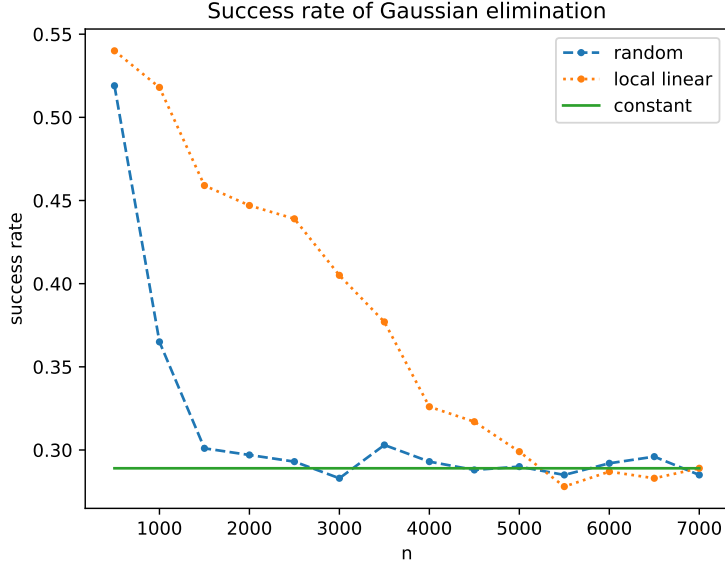


Fig. 1. Success rate of Gaussian elimination for the matrix $\mathbf{H}'$. The ‘constant’ line indicates the theoretical value of 0.289.

Table 14. The parameters and size of $\mathbf{W}'$ in our experiments.

$\text{RSD}(n, k, h)$	(f, u, g)	Size of $\mathbf{W}'$
$(200, 80, 25)$	$(15, 1, 3)$	198×171
$(200, 85, 25)$	$(15, 1, 3)$	311×276
$(300, 105, 25)$	$(15, 2, 2)$	419×378
$(400, 120, 25)$	$(20, 1, 2)$	1056×990
$(400, 135, 25)$	$(20, 1, 2)$	1859×1770
$(500, 150, 25)$	$(20, 1, 2)$	2310×2211
$(600, 170, 30)$	$(20, 2, 2)$	1983×1891
$(800, 200, 40)$	$(20, 2, 2)$	3443×3321
$(2^{10}, 652, 57)^*$	$(45, 10, 3)$	2310×2211
$(2^{12}, 1589, 172)^*$	$(169, 6, 3)$	19799×19503
$(2^{12}, 1589, 98)^*$	$(72, 13, 6)$	28559×28203

C.2 Testing the Elimination of $\mathbf{W}'$ and the Entire Algorithm

To evaluate the success rate of the partial Gaussian elimination of the matrix $\mathbf{W}'$ and test the entire algorithm, we strictly adhere to the experimental procedure outlined in [Section 3.1](#) and [Section 3.2](#). Specifically, we first generate RSD instances at random, then repeatedly guess the (f, u) error-free positions, perform Gaussian elimination on the reduced matrix, construct the quadratic equations, and finally enumerate the nonzero positions of w coefficients to solve the linear system. In our experiments, we use the parameter sets listed in [Table 14](#), where the RSD dimension n ranges from 200 to 4096, and the size of the corresponding matrix $\mathbf{W}'$ ranges from 198×171 to 28559×28203 ⁴.

For the first 8 parameter sets, we conduct over 100 experiments on the entire algorithm, with each RSD instance being randomly generated. For the last 3 parameter sets with *, since the complexity of whole hybrid algorithm is too large, we use some priori knowledge to accelerate and only verify the success of partial Gaussian elimination as well as the whole algorithm under the correct (f, u) guess and ‘coefficient’ enumeration. Specifically, we first generate RSD instances at random, then guess the **correct** (f, u) error-free positions, perform Gaussian elimination on the reduced matrix, construct the quadratic equations, and finally substitute the **correct** value of w coefficients to solve the linear system. From the results, we observe a 100% success rate for the partial Gaussian elimination of $\mathbf{W}'$. Furthermore, the entire algorithm successfully recover the error vector in all experiments for the selected parameters, confirming its effectiveness.

D Comparison with the ISD Approach in [\[31\]](#) without Truncation

To facilitate a more equitable comparison with the ISD approaches in [\[31\]](#), we adapt our algorithm to exclude the use of truncation and conduct an additional comparison. Specifically, we retain the last $n - h \lfloor n/h \rfloor$ variables of $\mathbf{e}$ unchanged during the error-free positions guess step, and consider these variables as ‘unknowns’ (i.e., contained in $(\tilde{e}_1, \dots, \tilde{e}_v)$ defined in [Section 3.1](#)) during the construction and solution of quadratic equations. The time complexity is detailed in [Table 15](#) and [Table 16](#).

⁴ Since we have not adopt the optimizations in the experiments, the choice of (f, u, g) is not the same as [Table 7](#) and [Table 8](#), which might introduce much more equations than we need. So we truncate m_1 to $\frac{v^2-v}{2} + \frac{5v}{2}$ to verify the procedures strictly.

Table 15. The time complexity for RSD in $\mathbb{F}_{2^{128}}$ *without truncation*.

RSD(n, k, h)	ISD in [31]	Ours	Parameters ($\bar{f}, \bar{u}, g$)
$(2^{22}, 64770, 4788)$	152	89.29	(1615,0,3)
$(2^{20}, 32771, 2467)$	155	114.62	(794,4,3)
$(2^{18}, 15336, 1312)$	153	127.10	(1238,5,3)
$(2^{16}, 7391, 667)$	151	134.73	(133,7,3)
$(2^{14}, 3482, 338)$	150	139.09	(317,7,2)
$(2^{12}, 1589, 172)$	155	146.37	(3,8,2)
$(2^{10}, 652, 106)$	194	189.24	(96,5,2)
$(2^{22}, 64770, 2735)$	108	86.71	(0,0,0)
$(2^{20}, 32771, 1419)$	107	86.63	(61,2,3)
$(2^{18}, 15336, 760)$	105	94.06	(506,6,3)
$(2^{16}, 7391, 389)$	103	98.14	(100,10,2)
$(2^{14}, 3482, 198)$	101	98.81	(2,12,2)
$(2^{12}, 1589, 98)$	100	97.29	(94,12,2)
$(2^{10}, 652, 57)$	111	109.82	(37,10,1)

Table 16. The time complexity for binary RSD *without truncation*.

RSD(n, k, h)	ISD in [31]	Ours	Parameters ($\bar{f}, \bar{u}, g$)
$(2^{22}, 64770, 4788)$	135	87.04	(0,0,3)
$(2^{20}, 32771, 2467)$	135	105.79	(360,3,4)
$(2^{18}, 15336, 1312)$	129	116.30	(1033,4,4)
$(2^{16}, 7391, 667)$	125	122.41	(31,6,4)
$(2^{14}, 3482, 338)$	122	124.31	(269,6,3)
$(2^{12}, 1589, 172)$	121	126.56	(150,6,3)
$(2^{10}, 652, 106)$	159	138.12	(90,4,8)
$(2^{22}, 64770, 2735)$	99	87.15	(0,0,3)
$(2^{20}, 32771, 1419)$	97	83.80	(52,1,3)
$(2^{18}, 15336, 760)$	92	89.83	(497,5,3)
$(2^{16}, 7391, 389)$	87	92.63	(318,8,3)
$(2^{14}, 3482, 198)$	83	92.08	(117,10,3)
$(2^{12}, 1589, 98)$	80	86.27	(55,13,6)
$(2^{10}, 652, 57)$	90	88.82	(40,9,6)